

АННОТАЦИЯ

Дипломная работа посвящена разработке виртуальной лаборатории — программно-аппаратной системы для безопасного удалённого управления группой роботов через интернет. Предложена и реализована архитектура «тонкое устройство — умный сервер»: вся содержательная логика (планирование, очереди команд, контроль доступа, координация группы, телеметрия, безопасность) сосредоточена на центральном сервере, а на борту робота остаётся лишь прозрачный сетевой мост и низкоуровневый контроллер. Разработан программный комплекс, включающий серверное ядро управления, сетевой слой на основе WebSocket с аутентификацией и клиентскую библиотеку как кроссплатформенный программный интерфейс доступа к лаборатории. Система поддерживает гетерогенную группу устройств (роботы на базе ROS и устройства с прямым обменом по последовательному порту), с возможностью масштабирования и добавления новых типов, многопользовательский доступ с разграничением прав, приоритетную диспетчеризацию команд с возможностью аварийного прерывания, автоматическое восстановление при сбоях и серверные групповые процедуры для скоординированных коллективных сценариев.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
Глава 1. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ.....	6
1.1. Существующие решения удалённого управления роботами	6
1.2. Сводное сравнение рассмотренных систем	10
1.3. Требования к системе управления группой роботов.....	10
Глава 2. АРХИТЕКТУРА СИСТЕМЫ	12
2.1. Общие архитектурные принципы.....	12
2.2. Сквозная схема системы: абстракция «транспорты и адаптеры»	14
2.3. Серверное ядро управления.....	17
2.4. Групповой характер лаборатории	18
2.5. Сетевой слой и безопасность системы	19
Глава 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	21
3.1. Организация серверного ядра: приоритетная диспетчеризация команд.....	22
3.2. Надёжная потоковая доставка команд и расширение системы	25
3.3. Контроль доступа и супервизор устройств.....	28
3.4. Групповые процедуры	30
3.5. Телеметрия и сетевой протокол обмена	33
Глава 4. ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА СИСТЕМЫ	36
4.1. Экспериментальный стенд	36
4.2. Количественные и качественные измерения	38
4.3. Навигационная процедура «По домам» для роботов uarq-13	40
4.4. Сводка результатов	41
ЗАКЛЮЧЕНИЕ	42
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	44
ПРИЛОЖЕНИЯ.....	45

ВВЕДЕНИЕ

Актуальность

Развитие робототехники сопровождается устойчивым ростом интереса к гетерогенным группам роботов, способным совместно действовать в распределённой среде, и к системам удалённого доступа к такому оборудованию. Виртуальные лаборатории, позволяющие наблюдать за реальными роботами и управлять ими через интернет, открывают новые возможности для исследований и обучения в области групповой робототехники: они повышают эффективность использования дорогостоящего оборудования, упрощают доступ к экспериментам и снимают необходимость в собственной аппаратной базе у каждого исследователя.

К настоящему времени предложено множество решений для удалённого управления роботами и коммерческих систем телеуправления. Однако анализ существующих разработок выявляет ряд общих ограничений: слабую поддержку гетерогенных систем, отсутствие единого протокола взаимодействия, ограниченные возможности многопользовательского доступа и проблемы масштабируемости. Главное же ограничение носит концептуальный характер: подавляющее большинство систем ориентировано на управление отдельными роботами, а не на работу с группой как с единым объектом.

Между тем групповые робототехнические системы обладают собственной спецификой, которую необходимо учитывать на уровне архитектуры: это одновременный доступ к состоянию всех роботов группы (координаты, параметры, телеметрия), поддержка межагентного взаимодействия, обеспечение безопасности (в том числе предотвращение столкновений) и выполнение групповых операций — синхронного старта, общей остановки, скоординированных коллективных сценариев. Отсутствие стандартизированных интерфейсов и механизмов безопасного распределения ресурсов между несколькими пользователями затрудняет проведение полноценных групповых экспериментов.

Таким образом, существующие решения недостаточны для работы с гетерогенными группами роботов: актуальной является разработка системы группового удалённого управления, изначально учитывающей особенности коллективного взаимодействия и обеспечивающей единообразную, безопасную и многопользовательскую работу с разнородной группой устройств.

Цель работы

Повышение эффективности экспериментальных исследований в области групповой робототехники путём создания системы управления виртуальной лабораторией с учётом специфики группового взаимодействия.

Задачи

Для достижения поставленной цели решаются следующие задачи:

1. Провести сравнительный анализ существующих решений в области удалённого управления роботами и веб-интерфейсов и сформировать на его основе требования к системе управления группой роботов.
2. Разработать архитектуру системы виртуальной лаборатории с поддержкой различных типов устройств и протоколов взаимодействия.
3. Реализовать алгоритмы и методы удалённого управления, соответствующие требованиям к подобным системам.
4. Разработать клиентский интерфейс для управления роботами с возможностью видеотрансляции, сбора телеметрии и отправки команд в реальном времени.
5. Провести серию испытаний системы на примере управления несколькими типами роботов и создать по её итогам методическое руководство.

Объект исследования: распределённые системы управления техническими объектами.

Предмет исследования: гетерогенные робототехнические системы с удалённым управлением.

Используемые методы

В работе применяются методы объектно-ориентированного программирования и разработки распределённых систем, сетевые технологии передачи данных, асинхронное программирование, а также аппарат теории автоматического управления.

Практическая значимость заключается в создании нового класса удалённых робототехнических лабораторий, способных проводить эксперименты с гетерогенными группами роботов при централизованной серверной координации и многопользовательском доступе.

Выносимые на защиту положения

1. Методы и алгоритмы серверного управления группой для безопасной многопользовательской работы в реальном времени: приоритетная диспетчеризация команд (независимые очереди устройств, аварийное прерывание, пауза на время

восстановления, синхронная многоадресная доставка) и надёжная потоковая доставка целевого состояния (setpoint) с фоновым «стоп-хвостом» по актуаторным каналам — гарантирующая безопасную остановку и устойчивость к потерям кадров на ненадёжном беспроводном канале.

2. Модель и алгоритмы централизованной групповой навигации (процедура «По домам»): кинематическая модель одноколёсника, многокамерное определение глобальных поз по ArUco-маркерам сигма-точечным фильтром Калмана (UKF), планирование маршрута алгоритмом A* по карте с препятствиями и waypoint-контроллер — обеспечивающие скоординированное приведение группы в целевые точки с обходом препятствий и устойчивостью к шуму и эпизодическому пропаданию измерений.
3. Реализация программного комплекса — серверное ядро (оркестратор RemoteLabManager с планировщиком команд, контролем доступа, супервизором устройств и менеджером групповых процедур), сетевой слой на WebSocket с аутентификацией и кроссплатформенная клиентская библиотека RemoteLab, — позволяющая организовать безопасное многопользовательское групповое управление разнородными устройствами через единый протокол.
4. Результаты экспериментальной проверки на физическом стенде (роботы uarp-13, позиционирование двумя камерами): удалённое управление в реальном времени, устойчивость к потерям кадров с гарантированной аварийной остановкой, автоматическое восстановление при разрывах связи и работа централизованной групповой навигации «По домам» на реальном оборудовании.

Глава 1. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

В этой главе рассматриваются существующие подходы к удалённому управлению роботами и веб-лабораториям. Цель обзора — выявить, какие архитектурные и функциональные решения уже найдены, а какие задачи остаются закрытыми лишь частично, и на этом основании сформулировать требования к разрабатываемой системе. Для анализа выбраны три показательные системы, охватывающие основные сложившиеся подходы: веб-доступ к одиночному роботу через ROS (rosbridge), безопасная роевая лаборатория удалённого доступа (Robotarium) и удалённая лаборатория для группы мобильных роботов с централизованным вычислителем (Remote Lab, Сиена). Завершают главу сводное сравнение систем и вытекающие из него требования.

1.1. Существующие решения удалённого управления роботами

Ниже последовательно разобраны три системы. По каждой приводится суть подхода, его сильные и слабые стороны и то, что из него заимствуется для настоящей работы.

Web Applications for Robots using rosbridge (Lee, 2012).

В 2012 году Jihoon Lee (Brown University) предложил архитектуру, объединяющую операционную систему роботов ROS с веб-технологиями для построения полноценных веб-приложений управления роботами [5]. ROS играет роль серверной части (обработка команд, драйверы, публикация данных датчиков); компонент rosbridge служит мостом между ROS и веб-клиентами, предоставляя унифицированный протокол поверх WebSocket с сериализацией сообщений в JSON, благодаря чему браузерный клиент получает доступ ко всем функциям ROS без локальной установки; библиотека rosjs даёт высокоуровневые методы работы с топиками и сервисами, а mjpeg_server транслирует видеопоток по HTTP. Подход продемонстрирован на ряде приложений для робота PR2 — от автономного планирования до управления на естественном языке и сценариев human-in-the-loop (Рисунок 1.1). Сильные стороны: кроссплатформенный доступ из любого браузера, низкий порог входа, гибкий протокол rosbridge с поддержкой нескольких клиентов. Ограничения существенны: не рассмотрены вопросы безопасности удалённого доступа (аутентификация, контроль прав, изоляция стороннего кода), MJPEG не обеспечивает низкой задержки видео, не проработаны многопользовательский режим и масштабирование. Для настоящей работы из этого решения берётся ключевая идея — лёгкий открытый протокол поверх WebSocket и использование ROS как унифицирующего middleware.

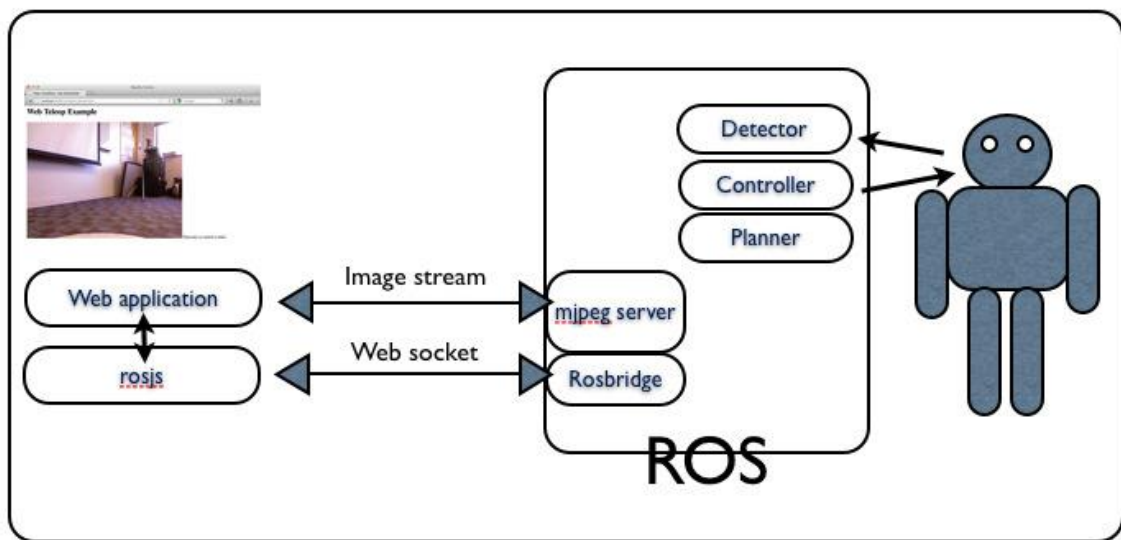


Рисунок 1.1 Архитектура системы rosbridge

Safe, Remote-Access Swarm Robotics Research on the Robotarium (Pickem et al., 2017).

Robotarium — удалённо доступная многоагентная лаборатория для исследований и образования в области роевой робототехники [8]. Платформа состоит из инфраструктуры лаборатории (парк мини-роботов GRITSBot дифференциального привода, глобальное позиционирование верхней камерой по ArUco-меткам, автоматическая зарядка, беспроводное перепрограммирование) и веб-интерфейса с серверным бэкендом. Пользователь через Matlab-API или браузер загружает алгоритм, который сначала проходит симуляцию, затем проверку безопасности и лишь потом исполняется на реальных роботах (Рисунок 1.2). Центральный вклад работы — двухуровневый механизм безопасности: симуляционная верификация (проверка на коллизии и выдача «оценки безопасности» до запуска) и онлайн-барьеры (barrier certificates), при которых пользовательский управляющий сигнал минимально корректируется так, чтобы система оставалась в множестве безопасных состояний — например, при соблюдении минимального расстояния между роботами и границ арены (Рисунок 1.3). Для настоящего проекта значимы проектирование с учётом удалённого доступа, обеспечение безопасности оборудования при исполнении стороннего кода и многопользовательский доступ с очередью; идея барьерных сертификатов прямо перекликается с рассматриваемой далее перспективой серверного контроля рабочей зоны.

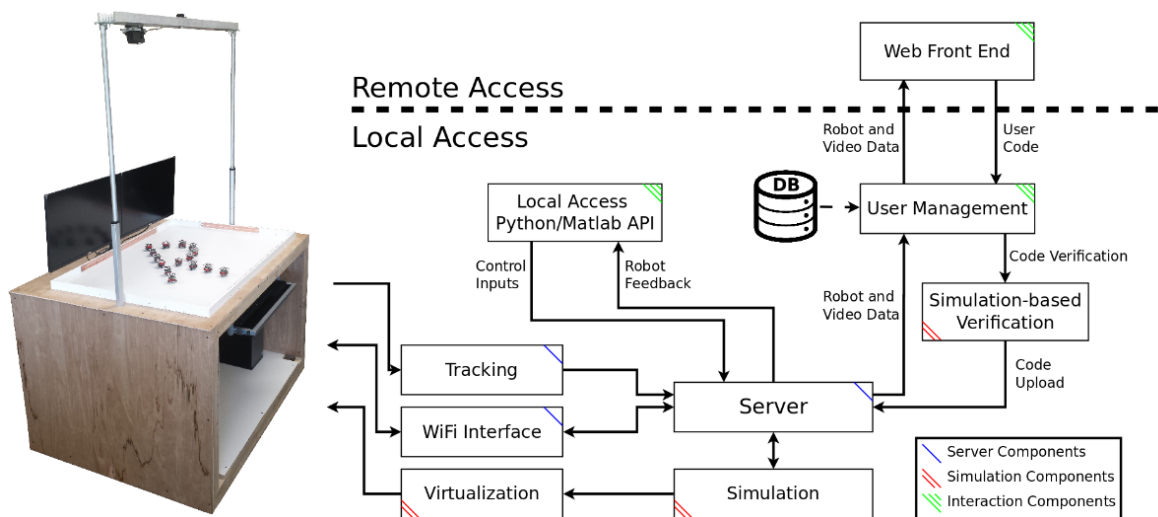


Рисунок 1.2 Архитектура Robotarium

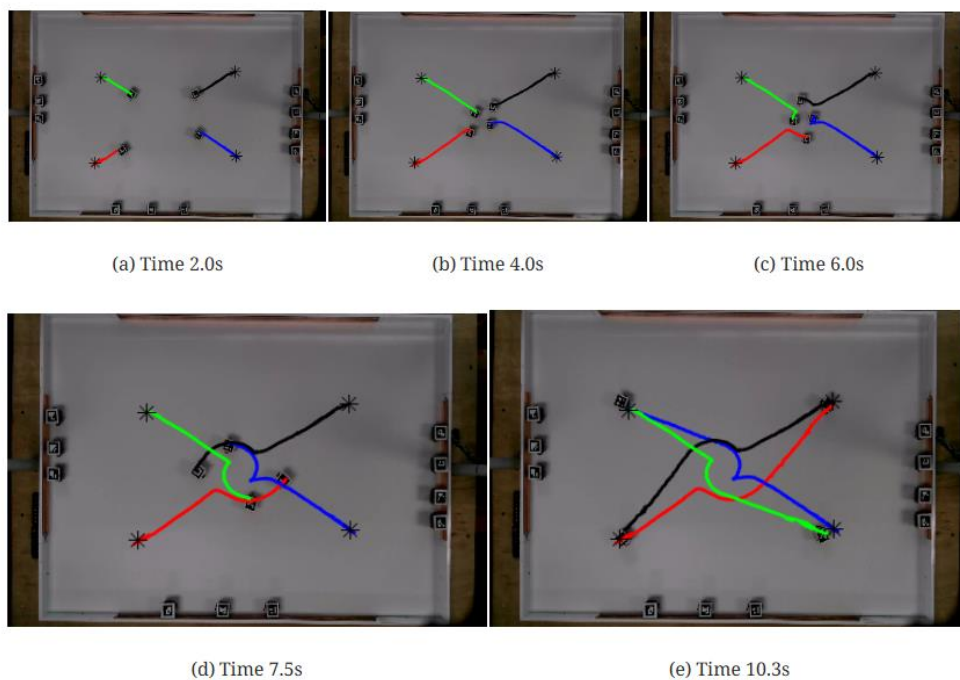


Рисунок 1.3 Демонстрация работы барьеров безопасности

A Remote Lab for Experiments with a Team of Mobile Robots (Casini et al., 2014).

Удалённая лаборатория Университета Сиены для экспериментов с группой мобильных роботов, встроенная в платформу дистанционного обучения АСТ [4]. Стенд — четыре одинаковых робота на базе LEGO Mindstorms NXT в рабочем пространстве 4.5×3 м (Рисунок 1.4); каждый робот описывается кинематической моделью одноколёсника, а на борту выполняется лишь низкоуровневый ПИ-регулятор скорости колёс. Глобальные позиция и ориентация роботов определяются двумя потолочными камерами по маркерам, по завершении эксперимента роботы автоматически приводятся на зарядку. Сервер

реализован как Matlab-функция: на каждом такте она получает позы всех роботов и возвращает желаемые линейную и угловую скорости, связывается с роботами по Bluetooth, передаёт видео и данные в интерфейс и обеспечивает автоматический останов при выходе робота за рабочую зону или сбое связи. Поддерживаются виртуальные препятствия и программно моделируемые сенсоры поверх точных поз от технического зрения, что позволяет испытывать децентрализованные стратегии. Для настоящей работы это решение наиболее показательное сразу по нескольким причинам: безопасность как обязательный модуль, симуляция перед исполнением, программное моделирование сенсоров поверх точной информации о позах и — главное — централизация вычислений на сервере при «тонком» бортовом уровне. Ограничения — привязка к среде Matlab и Java-апплету (снижающая кроссплатформенность) и ориентация на единственный тип роботов.

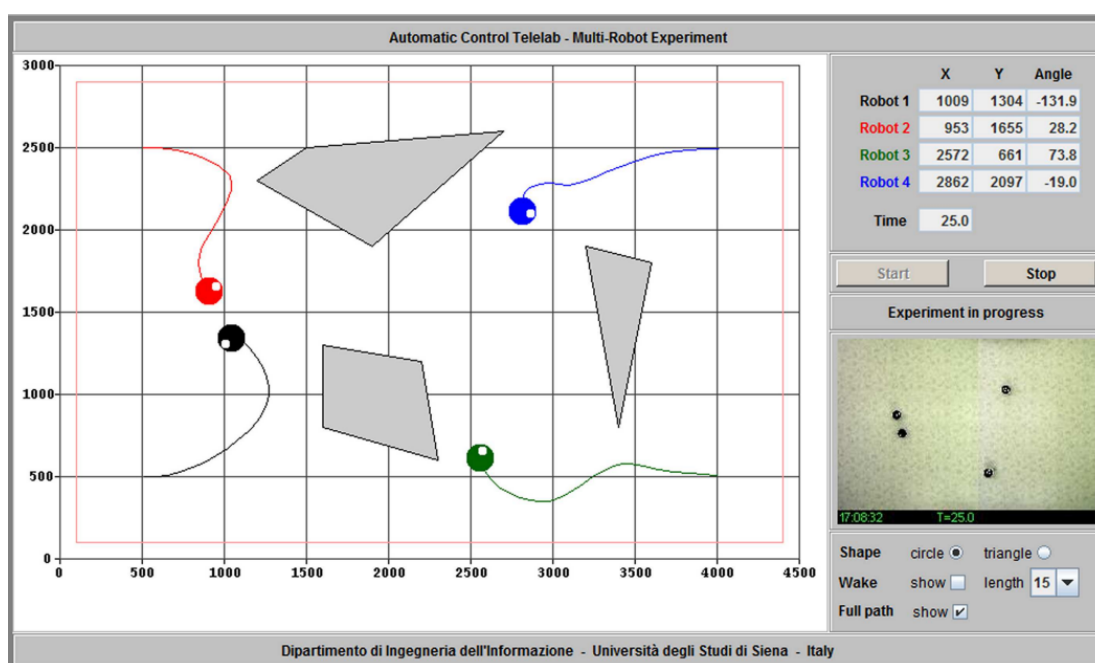


Рисунок 1.4 Пользовательский интерфейс АСТ

1.2. Сводное сравнение рассмотренных систем

Ключевые характеристики трёх систем сведены в таблицу по признакам, существенным для разрабатываемой лаборатории.

Характеристика	rosbridge [Lee, 2012]	Robotarium [Pickem et al., 2017]	Remote Lab Siena [Casini et al., 2014]
Класс роботов	одиночный	группа (рой)	группа
Гетерогенность	любой ROS-робот	однотипные	один тип (LEGO NXT)
Middleware / основа	ROS + rosbridge	собственный сервер + Matlab API	Matlab-сервер + Java-апплет
Протокол клиент-сервер	WebSocket (JSON)	загрузка кода (Matlab/веб)	TCP (Matlab - Java)
Где исполняется логика	сервер (ROS)	сервер (код)	сервер (Matlab)
Бортовые вычисления	зависят от робота	минимальные	только ПИ-регулятор
Позиционирование	—	камера + ArUco	2 камеры + маркеры
Видеопоток	да (MJPEG)	да	да (кадры с камер)
Многопольз. доступ	не рассмотрен	очередь / расписание	ограниченный
Безопасность	не рассмотрена	верификация + barrier certs	стоп при выходе за зону / сбое
Открытость / стандарты	высокая (ROS)	частичная	низкая (Matlab/Java)
Ключевой вклад	веб-доступ к ROS	безопасность роя	централизация + «тонкий» борт

Каждая из систем закрывает лишь часть требований к удалённой лаборатории, но ни одна не покрывает их в нужном сочетании. От rosbridge берётся идея лёгкого открытого протокола поверх WebSocket и использование ROS как унифицирующего middleware; от Robotarium — приоритет безопасности и многопользовательского доступа с очередью; от Remote Lab Siena — централизация вычислений при «тонком» бортовом уровне и обязательный модуль безопасного останова. При этом во всех трёх системах слабо проработаны либо гетерогенность роботов, либо многопользовательский режим, либо открытость интерфейса — что и формирует нишу разрабатываемой системы.

1.3. Требования к системе управления группой роботов

На основе проведённого анализа сформулированы ключевые требования к виртуальной лаборатории управления группой роботов.

1. **Доступность и кроссплатформенность.** Доступ к лаборатории с разных платформ (Windows, Linux, macOS) через стандартный сетевой интерфейс, без привязки к конкретной ОС или среде разработки.

2. **Многопользовательский доступ.** Авторизация пользователей и разграничение доступа к устройствам, предотвращение конфликтов при одновременной работе нескольких операторов.
3. **Унифицированная работа с разнородными роботами.** Абстракция над типами устройств и открытые протоколы обмена, позволяющие единообразно управлять гетерогенной группой и легко добавлять новые устройства.
4. **Реальное время и обратная связь.** Передача телеметрии и состояния роботов (координаты, параметры, заряд) в реальном времени.
5. **Групповое управление.** Работа с группой роботов как с единым объектом: групповые команды и скоординированные коллективные сценарии, а не только управление отдельными роботами.
6. **Безопасность и надёжность.** Аварийная остановка (E-stop) по требованию оператора, контроль состояния оборудования, логирование, автоматическое восстановление при сбоях.
7. **Масштабируемость и модульность.** Модульная архитектура, расширяемая новыми роботами, датчиками и интерфейсами без глобальной переработки системы.
8. **Совместимость и открытость.** Опора на открытые решения и свободные библиотеки, чётко описанные API и документация, позволяющие сторонним разработчикам расширять платформу.

Таким образом, анализ существующих решений показал, что ни одна из рассмотренных систем не обеспечивает одновременно гетерогенность, многопользовательский доступ и открытость интерфейса при работе с группой роботов, и на основе этого анализа сформулированы требования к разрабатываемой лаборатории. Далее, в Главе 2, на основе этих требований последовательно строится архитектура системы.

Глава 2. АРХИТЕКТУРА СИСТЕМЫ

Настоящая глава посвящена архитектуре разработанной системы. Сначала кратко обосновывается выбор базового подхода к организации связи с роботами — отказ от собственного формата обмена в пользу зрелого стека и вынос вычислений с борта робота на центральный сервер. Затем подробно описывается итоговая архитектура «тонкое устройство — умный сервер»: сквозной путь данных, состав серверного ядра, групповой характер лаборатории и встроенные механизмы безопасности. Глава отвечает на вопрос «как устроена система и почему именно так»; её программная реализация рассматривается в Главе 3. Архитектура системы в целом и её программные подсистемы спроектированы и реализованы автором в рамках настоящей работы.

2.1. Общие архитектурные принципы

Разработка начиналась с попытки построить собственный низкоуровневый протокол обмена между Wi-Fi-модулем ESP и контроллером Arduino. Однако анализ [Приложение: Низкоуровневый протокол связи] показал, что «пользовательский» (самодельный) формат транспортного уровня непригоден для долгоживущей масштабируемой системы: он статичен (любое новое сообщение требует одновременной перепрошивки всего парка устройств), не стандартизирован и лишён штатных средств диагностики и подтверждения доставки. Те же по сути ограничения свойственны и жёстким промышленным форматам вроде Modbus (фиксированные 16-битные регистры и однобитовые флаги, невозможность передать переменное число аргументов и сложные типы данных). Вывод из этого анализа однозначен: вместо изобретения собственного формата обмена целесообразно опереться на зрелый стандартизированный стек — операционную систему роботов ROS [9], которая предоставляет унифицированную модель обмена (топики, сервисы, типизированные сообщения), богатый инструментарий диагностики и развитую экосистему.

Принятие ROS ставит вопрос о том, где физически исполняется ROS-узел, обслуживающий конкретного робота. Первый очевидный ответ — разместить вычислитель прямо на борту, установив на каждого робота одноплатный компьютер (например, Raspberry Pi) с полноценным экземпляром ROS. Этот вариант архитектурно «чист», но был отвергнут по совокупности практических причин, критичных именно для лаборатории с парком из множества роботов: высокая стоимость в пересчёте на парк, заметные энергопотребление и масса, сложность обслуживания (каждый робот превращается в отдельную Linux-машину со своей файловой системой и версиями ПО) и

хрупкость распределённой ROS-сети поверх нестабильного WiFi. Функционально же большинство роботов — это «простые» машины, которым нужно лишь принимать команды движения и отдавать телеметрию, и для них полноценный бортовой компьютер избыточен.

Ключевое наблюдение, приведшее к итоговой архитектуре, состоит в следующем: роботу не нужны бортовые вычисления как таковые — для дистанционного управления от устройства требуется лишь подключиться к сети и передавать поток байтов между сетью и низкоуровневым контроллером в обе стороны. Поэтому бортовой вычислитель можно полностью убрать, а его функцию «моста» поручить дешёвому Wi-Fi-микроконтроллеру ESP, прошивка которого реализует прозрачный мост «последовательный порт ↔ TCP»: модуль не интерпретирует данные, а лишь пересылает байты между UART и открытым TCP-соединением. Вся «интеллектуальная» часть — ROS, логика команд, очереди, координация — переносится на центральный сервер; с точки зрения серверной ROS-ноды всё выглядит так, будто контроллер подключён к серверу напрямую по кабелю (Рисунок 2.1). Именно отказ от «интеллекта» на борту делает устройство проще, дешевле и надёжнее: вместо хрупкой распределённой ROS-сети — единственный ROS master на сервере и набор простых TCP-туннелей, а добавление новой команды или типа устройства не требует перепрошивки парка. Этот принцип — «тонкое устройство — умный сервер» — и составляет концептуальное ядро итоговой системы. В реализации в роли моста используется модуль ESP-32: по сравнению с ESP8266 он обладает двухъядерным процессором и более стабильным сетевым стеком, сохраняя ту же тривиальную роль «передатчика байтов».

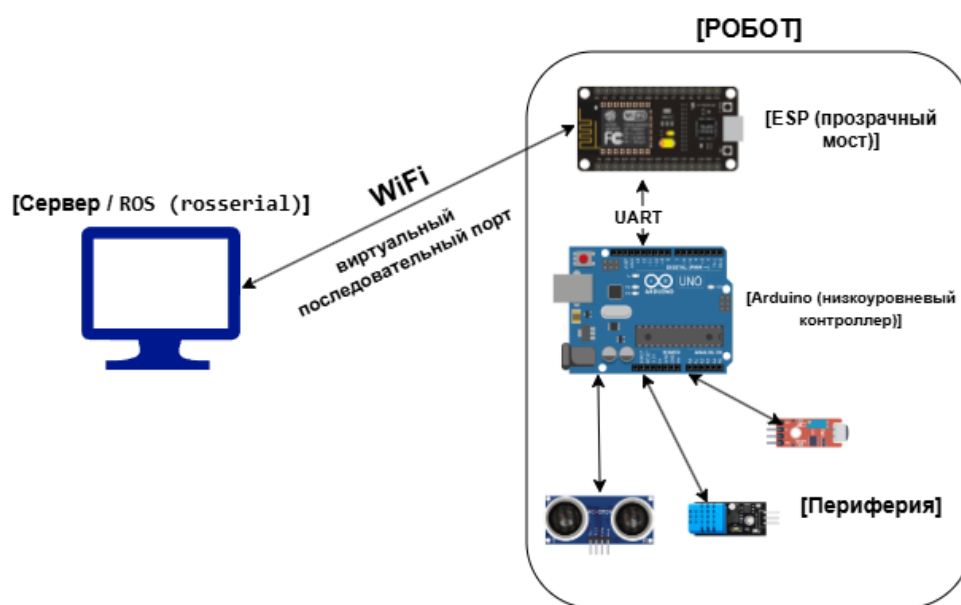


Рисунок 2.1 Схема «ROS+ESP»

В основе архитектуры, предлагаемой в данном исследовании, положен принцип «тонкое устройство — умный сервер»: на стороне робота остаётся лишь минимальная аппаратура (прозрачный сетевой мост и низкоуровневый контроллер), а вся содержательная логика — управление, очереди команд, контроль доступа, координация группы, телеметрия и безопасность — сосредоточена на центральном сервере. Такой подход напрямую отвечает требованиям из Главы 1: единая точка управления упрощает многопользовательский доступ и обеспечение безопасности, а дешёвое унифицированное устройство — масштабирование парка и работу с разнородными роботами.

Две черты итоговой архитектуры определяют её назначение. Во-первых, система изначально проектировалась как **групповая**: её предмет — управление не отдельным роботом, а группой устройств; это отражено на всех уровнях, от параллельных независимых очередей команд для каждого устройства до серверных групповых процедур над всей группой. Именно централизация логики на сервере, имеющем одновременный доступ к состоянию и очередям всех устройств, делает возможной настоящую групповую координацию, а не просто параллельное управление несколькими роботами по отдельности. Во-вторых, система **не привязана к одному типу устройств и одному стеку**: ROS — основная и наиболее проработанная спецификация взаимодействия, но не ограничение архитектуры; за счёт абстракции драйверов поддерживаются и устройства, не использующие ROS (в частности, уже реализован драйвер для устройства с прямым обменом по последовательному порту).

2.2. Сквозная схема системы: абстракция «транспорты и адаптеры»

Полный путь данных от пользователя до исполнительных механизмов робота включает несколько уровней (Рисунок 2.2): клиентская библиотека → сетевой слой сервера → серверное ядро (контроль доступа, планировщик, драйверы) → транспорт и адаптер → сетевой мост на роботе → низкоуровневый контроллер; в обратную сторону тем же путём проходит телеметрия. Ниже последовательно описаны уровни этой архитектуры.

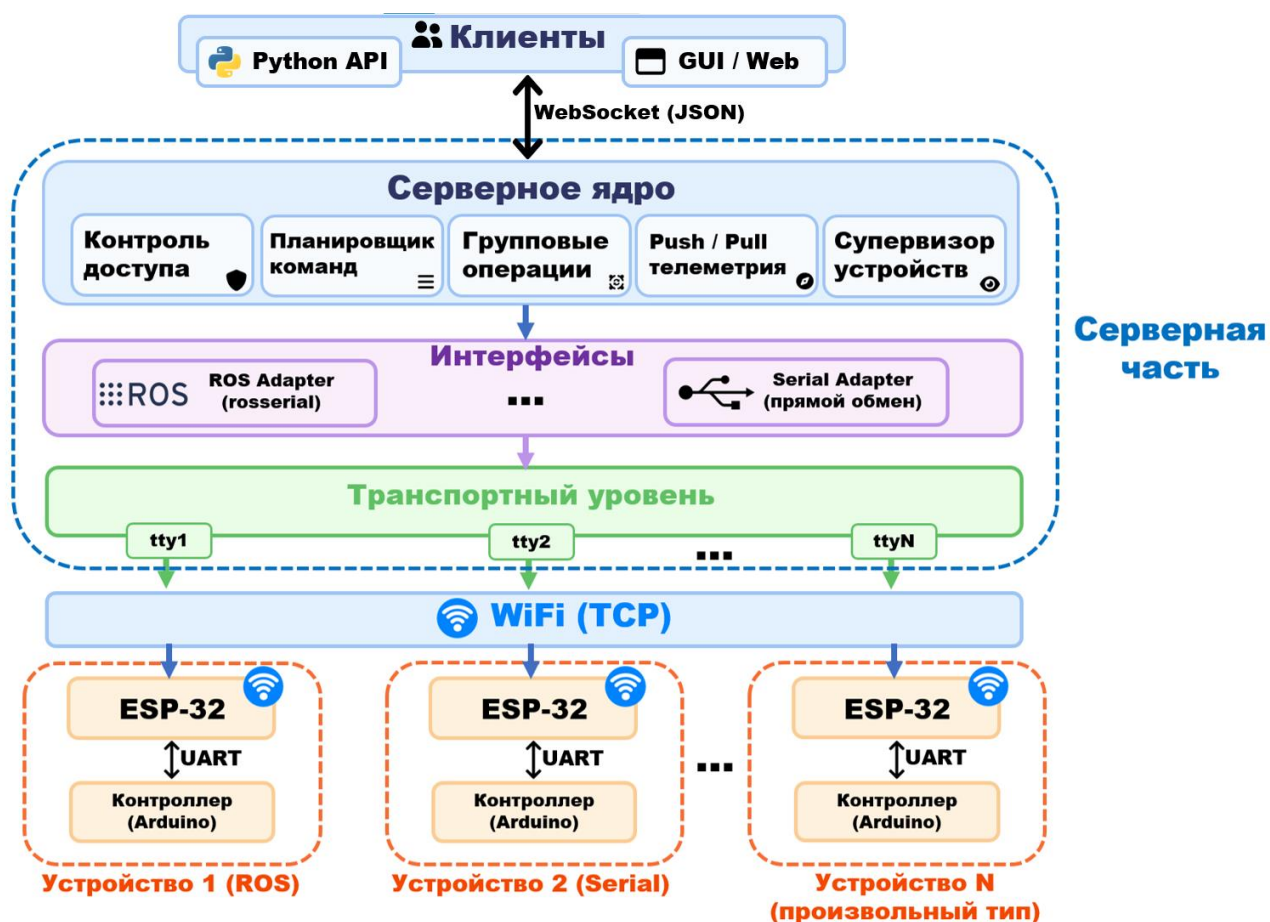


Рисунок 2.2 Архитектурные уровни

Тонкое устройство

На борту робота находятся только два компонента. Сетевой мост ESP-32 реализует прозрачный канал «TCP ↔ UART» [Приложение: Полный код ESP-32 (мост TCP ↔ UART)]: байты, пришедшие из TCP-соединения по WiFi, передаются по UART низкоуровневому контроллеру, и наоборот. Прошивка не интерпретирует содержимое и не меняется при добавлении новых команд или типов устройств. Низкоуровневый контроллер (Arduino) исполняет встроенный клиент roserial: принимает по UART сообщения, преобразует их в непосредственные управляющие воздействия (ШИМ моторов, команды сервоприводам) и публикует обратно показания датчиков. Важно, что контроллер «не знает» о существовании сети — с его точки зрения он соединён с ROS-системой обычным последовательным портом; сеть полностью инкапсулирована в паре «ESP-32 (на роботе) ↔ виртуальный порт socat (на сервере)».

Транспортный конвейер (socat)

Задача транспортного уровня — «вернуть» удалённое устройство на сервер так, будто оно подключено локально по последовательному порту. Эту функцию выполняет утилита socat, которая для каждого устройства создаёт виртуальный последовательный

порт (псевдотерминал, `pty`) с уникальным именем и связывает его с TCP- соединением до ESP-32 робота, изолируя устройства друг от друга. Соединение устанавливается с параметрами повышения надёжности (отключение алгоритма Нейгла для снижения задержек, автоматические повторные попытки подключения), что важно при работе по нестабильному WiFi.

Адаптерный конвейер (`rosserial`)

После того как удалённое устройство представлено на сервере виртуальным последовательным портом, к нему подключается `rosserial`, транслирующий поток сообщений из порта в полноценные топики ROS и обратно. Каждое устройство получает собственное пространство имён ROS, за счёт чего топики разных роботов не пересекаются. С этого момента устройство — обычный участник ROS-сети на сервере: команды публикуются в его топики, а телеметрия читается из топиков датчиков.

Абстракция «транспорты и адаптеры»

Связка `socat` + `rosserial` — частный, хотя и основной, случай более общей идеи: каждый драйвер устройства объявляет набор необходимых ему вспомогательных процессов — транспортов (средства доставки байтов, например `socat`) и адаптеров (средства превращения байтового потока в высокоуровневый интерфейс, например `rosserial`). Именно эта абстракция снимает привязку системы к ROS: драйвер сам решает, какие вспомогательные процессы ему нужны и как трактовать поток данных. Так, ROS-устройство (например, робот `uagp-13`) использует и транспорт, и адаптер и выражает команды через топики ROS, а простое последовательное устройство использует только транспорт и работает с байтовым потоком напрямую, без ROS. Тем самым ключевой принцип системы можно сформулировать так: сервер переводит унифицированные команды пользователя в команды, понятные родному контроллеру конкретного устройства, — и именно в этом переводе достигается поддержка разнородной группы. Архитектура не привязана ни к конкретной модели робота (не только `uagp-13`), ни к ROS как обязательному стеку, ни к `socat` как единственному способу доставки байтов: поскольку транспорт сменный, для моста на базе ESP-32 принципиально возможны и иные каналы связи — например, Bluetooth, которым модуль оснащён. Новый тип робота или интерфейса добавляется написанием драйвера на сервере, без изменения прошивки устройств. Программная реализация драйверного слоя подробно описана в Главе 3.

2.3. Серверное ядро управления

Над уровнем аппаратных интерфейсов располагается ядро системы — оркестратор RemoteLabManager, объединяющий несколько подсистем; их состав и связи показаны на схеме архитектуры сервера (Рисунок 2.3). Все перечисленные подсистемы серверного ядра разработаны и реализованы автором; подробное описание их алгоритмов приводится в Главе 3.

- Драйверы устройств — преобразуют абстрактные команды («двигаться», «сигнал», «повернуть сервопривод») в конкретные воздействия (публикации в топики ROS либо строки в последовательный порт) и отвечают за корректную физическую остановку устройства при прерывании.
- Планировщик команд (CommandScheduler) — ведёт для каждого устройства отдельную очередь с приоритетами и исполняет команды последовательно; поддерживает прерывание текущей команды, что служит основой механизма аварийной остановки.
- Контроль доступа (AccessController) — разграничивает монопольный и общий доступ к устройствам между несколькими клиентами.
- Супервизор устройств (DeviceSupervisor) — отслеживает состояние вспомогательных процессов и перезапускает их при сбоях, приостанавливая на время восстановления соответствующую очередь команд.
- Менеджер групповых процедур (ProcedureManager) — исполняет серверные сценарии над группой устройств.

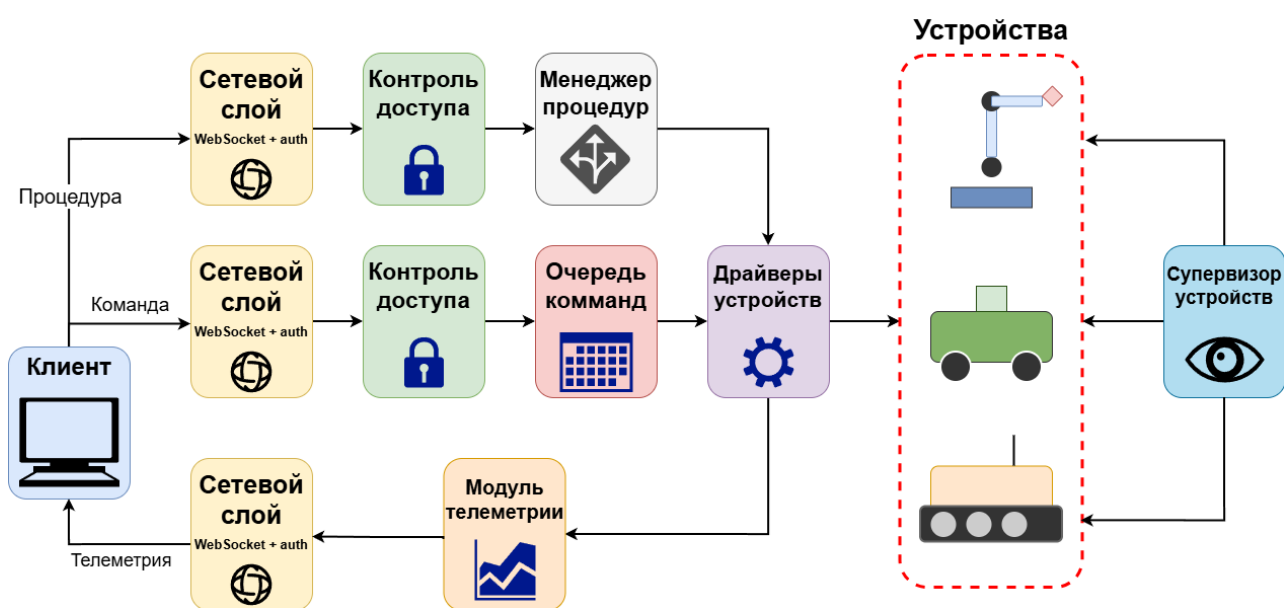


Рисунок 2.1 Архитектура серверного ядра

2.4. Групповой характер лаборатории

Групповой аспект — определяющая черта системы, и архитектурно он реализован на трёх уровнях нарастающей координации. Параллельное независимое управление: каждое устройство обслуживается собственной очередью команд и отдельным рабочим процессом, поэтому роботы управляются параллельно и независимо, а в сочетании с контролем доступа несколько операторов могут одновременно работать с разными роботами одной лаборатории. **Команды над подмножеством группы:** одна команда может быть адресована сразу нескольким устройствам — система ставит её в очередь каждого независимо и сигнализирует о завершении лишь тогда, когда команду исполнили все адресаты, что даёт простой способ синхронных групповых действий (одновременный старт, общая остановка). **Групповые процедуры** — серверные коллективные сценарии — это ключевой механизм именно скоординированного поведения: процедура выполняется на сервере, имеющем доступ к телеметрии и очередям всех устройств, и может выдавать команды множеству роботов, дожидаться их физического выполнения, читать телеметрию и принимать решения централизованно. В отличие от одиночных команд, процедура кодирует многошаговую логику с состоянием, охватывающую всю группу, и потому способна реализовать подлинную централизованную многоагентную координацию. Групповые процедуры, в том числе навигационная процедура «По домам», реализованы автором.

Показательный пример — процедура «По домам» (**AllGoHome**), приводящая всю группу роботов в назначенные домашние точки: каждый робот прокладывает маршрут из своей текущей позиции и проезжает его, объезжая препятствия, тогда как координацию всей группы централизованно осуществляет сервер. Идейно контур работы процедуры образуют четыре связанных компонента, объединённых серверным циклом:

1. **восприятие** — глобальные позы роботов определяются системой технического зрения (закреплённые над рабочим полем камеры фиксируют ArUco-маркеры на роботах; этот же приём применён в рассмотренных в Главе 1 лабораториях Robotarium и Сиены);
2. **оценка состояния** — зашумлённый и эпизодически пропадающий поток измерений сглаживается фильтром, дающим устойчивую оценку позы;
3. **планирование** — для каждого робота строится маршрут по карте рабочего пространства с обходом препятствий;
4. **управление движением** — робот ведётся по маршруту, причём управляющие воздействия выдаются не в обход системы, а через штатный конвейер команд лаборатории.

Поскольку решение принимается централизованно и одновременно для всей группы, на основе общего поля зрения камер и общей карты препятствий, AllGoHome реализует подлинно коллективный, скоординированный сценарий, а не простое параллельное выполнение независимых команд. Прикладная логика процедуры предварительно отработана автором на отдельном симуляторе навигации [7]. Формальная постановка контура (модель движения, фильтр, планировщик, контроллер) и его программная реализация приведены в Главе 3.

2.5. Сетевой слой и безопасность системы

Внешний доступ к системе обеспечивает сетевой слой на базе FastAPI/uvicorn. Взаимодействие с клиентом построено поверх постоянного WebSocket-соединения; формат обмена — текстовые JSON-сообщения с типовым дискриминатором (команды, подтверждения, события завершения, телеметрия, ошибки). Перед установлением соединения выполняется аутентификация. Со стороны пользователя работа ведётся через клиентскую библиотеку на Python (RemoteLab), разработанную автором и скрывающую детали протокола за высокоуровневым асинхронным интерфейсом: получение списка устройств, захват и освобождение устройства, отправка команд с ожиданием их выполнения, подписка на телеметрию, запуск групповых процедур. Библиотека самостоятельно поддерживает соединение, переподключаясь при обрывах с восстановлением захватов и подписок. Выбор формы клиента в виде библиотеки, а не веб-интерфейса, обеспечивает кроссплатформенность и возможность программной интеграции лаборатории в пользовательские сценарии и эксперименты.

Чтобы связать описанные уровни воедино, проследим прохождение данных в обе стороны.

Путь команды (от пользователя к роботу): пользователь вызывает метод библиотеки, и команда уходит на сервер JSON-сообщением по WebSocket; сетевой слой проверяет права доступа и передаёт команду планировщику, который ставит её в очередь нужного устройства согласно приоритету; рабочий процесс устройства извлекает команду и вызывает драйвер, публикующий соответствующее сообщение; roserial сериализует его в виртуальный последовательный порт, socat отправляет байты по TCP на ESP-32, а тот по UART — на Arduino, который исполняет команду.

Путь телеметрии (от робота к пользователю) проходит тем же маршрутом в обратную сторону: Arduino публикует показания датчиков, поток через ESP-32 → socat → roserial

превращается в топик ROS, драйвер на сервере формирует структурированный снимок телеметрии и уведомляет подписчиков, а сетевой слой пересылает снимок клиенту, где библиотека вызывает пользовательский callback. Программная реализация каждого из этих компонентов описана в Главе 3.

Безопасность системы

Поскольку система обеспечивает удалённое управление реальными роботами через сеть, вопросы безопасности рассматривались как неотъемлемая часть архитектуры. Их удобно разделить на два аспекта: защиту самой системы от несанкционированного доступа и обеспечение безопасности физических действий роботов.

Защита от несанкционированного доступа. Управление физическим оборудованием не должно быть доступно постороннему, поэтому доступ к системе закрыт аутентификацией: проверка учётных данных выполняется уже на этапе установления соединения (WebSocket-рукопожатия), и неаутентифицированное соединение отклоняется, не получая возможности отправлять команды. Пароли хранятся не в открытом виде, а в виде криптографических хешей с «солью» (схема PBKDF2 [2]), что защищает их при компрометации хранилища. Дополнительный уровень разграничения обеспечивает контроль доступа: монопольный захват устройства не позволяет одному пользователю вмешиваться в работу робота, которым управляет другой, а привязка захватов к сессии гарантирует освобождение ресурсов при отключении клиента. При развёртывании в недоверенной сети транспорт WebSocket дополнительно может быть закрыт шифрованием (TLS), что не требует изменения логики системы.

Безопасность физических действий роботов. Вторая, не менее важная сторона — предотвращение опасного поведения самого оборудования; здесь безопасность обеспечивается несколькими независимыми механизмами, реализованными в настоящей работе. Аварийная остановка (E-stop): оператор может одной операцией остановить как отдельное устройство, так и всю группу — планировщик очищает очереди и прерывает исполняемые команды, после чего драйверы гарантированно переводят устройства в состояние покоя. Надёжная остановка на ненадёжном канале: даже при потере кадров команда остановки доводится до устройства потоковой доставкой со стоп-хвостом, в том числе при аварийном прерывании (механизм описан в Главе 3). Отказоустойчивость: при сбое связи или вспомогательных процессов супервизор переводит очередь устройства в безопасное ожидание и восстанавливает его, не теряя команд. Бортовой контроль связи: низкоуровневый контроллер робота отслеживает поступление команд и при потере связи с

сервером самостоятельно останавливает движение, что служит последним рубежом защиты независимо от состояния сети и сервера.

Перспектива: серверный мониторинг безопасности. Принятая архитектура с централизацией логики на сервере открывает возможность для более развитых механизмов безопасности, естественных именно для групповой лаборатории. Поскольку сервер централизованно располагает картиной состояния всей группы — в том числе глобальными позами роботов от системы технического зрения (теми же камерами и ArUco-маркерами, что используются процедурой «По домам»), — на сервере может быть реализован программный «надзор» за безопасностью рабочей зоны: непрерывный контроль положения роботов с принудительной остановкой или коррекцией траектории тех из них, кто приближается к границе полигона, к препятствиям или к другим роботам. Такой механизм программного барьера концептуально близок к барьерным сертификатам платформы Robotarium, рассмотренной в Главе 1, и является естественным направлением развития системы, опирающимся на уже имеющуюся в ней централизованную модель состояния группы.

Таким образом, в главе обоснована и описана архитектура «тонкое устройство — умный сервер»: вычисления вынесены с борта робота на центральный сервер, разнородные устройства унифицированы драйверным слоем, групповой характер заложен на всех уровнях, а вопросы доступа и физической безопасности проработаны как часть архитектуры. Далее, в Главе 3, рассматривается программная реализация этих принципов — конкретные алгоритмы, структуры данных и механизмы серверного ядра.

Глава 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Глава 2 описала архитектуру системы — её уровни, принцип «тонкое устройство — умный сервер» и путь данных от пользователя до исполнительных механизмов робота. Настоящая глава посвящена программной реализации серверного ядра: конкретным алгоритмам, структурам данных и программным механизмам, разработанным в работе для воплощения этой архитектуры. Если предыдущая глава отвечала на вопрос «из чего состоит система и почему она устроена именно так», то данная — на вопрос «как именно это реализовано». Для каждой подсистемы приводится её модель (формальная или качественная), алгоритм работы и — для ключевых мест — псевдокод; полные исходные тексты программ, на которые нет смысла приводить целиком ввиду их объёма, вынесены в приложение, и на них даны ссылки.

Серверное ядро реализовано на языке Python в рамках единой асинхронной модели (библиотека `asyncio`). Выбор асинхронной модели неслучаен: сервер одновременно обслуживает множество источников событий — сетевые соединения нескольких клиентов, потоки телеметрии от всех устройств, фоновые процессы транспортов и адаптеров, параллельные очереди команд и долгоживущие групповые процедуры. Кооперативная многозадачность `asyncio` позволяет вести их в одном процессе без явных блокировок и потоков ОС.

3.1. Организация серверного ядра: приоритетная диспетчеризация команд

Все подсистемы объединены оркестратором `RemoteLabManager` — верхнеуровневым компонентом, который владеет остальными модулями и связывает их между собой. Такая организация со строгим единым владельцем выбрана, чтобы исключить скрытые зависимости между подсистемами и сосредоточить управление их жизненным циклом в одной точке. В состав ядра входят аппаратные интерфейсы (`RosInterface` — публикация и подписка на топики ROS, и `SerialInterface` — асинхронный обмен по последовательному порту), общие для всех драйверов соответствующего типа; фабрика драйверов `DriverFactory`, создающая для каждого устройства драйвер нужного типа по его описанию в конфигурации; планировщик команд, контроль доступа, супервизор устройств и менеджер групповых процедур (каждый рассмотрен в отдельном разделе ниже); а также сетевой слой, реализующий протокол обмена с клиентами.

Конфигурация лаборатории (перечень устройств, их адреса, типы драйверов, режим доступа) задаётся декларативно во внешнем файле и считывается при запуске; на основе этого описания оркестратор создаёт драйверы и регистрирует каждое устройство во всех подсистемах. Запуск системы выполняется в строго определённом порядке: сначала супервизор поднимает вспомогательные процессы устройств (транспорты и адаптеры) и инициализирует телеметрию, и лишь затем запускаются рабочие процессы планировщика, принимающие команды, — так что к моменту приёма первой команды все устройства уже представлены на сервере и готовы к работе. Завершение выполняется в обратном порядке с корректным освобождением сетевых и аппаратных ресурсов. Общая структура серверного ядра и связи между его модулями приведены на схеме архитектуры сервера в Главе 2 (Рисунок 2.3).

Приоритетная диспетчеризация команд

Решаемая задача. Лаборатория многопользовательская, поэтому в каждый момент времени к устройству может поступать несколько команд — от разных операторов и от групповых процедур. При этом физическое устройство способно выполнять команды только последовательно (нельзя одновременно «ехать вперёд» и «развернуться на месте»), а отдельные команды — в первую очередь аварийная остановка — должны иметь возможность опередить остальные. Для согласования этих требований в работе разработан планировщик команд. Его устройство удобно описать через структуру данных, порядок исполнения и механизмы прерывания, паузы и многоадресности.

Структура. Каждому устройству выделяется собственная приоритетная очередь команд и отдельный рабочий процесс-корутина (worker), который последовательно извлекает из неё команды и передаёт их драйверу на исполнение. За счёт того, что у каждого устройства свой независимый worker, роботы управляются параллельно: команда одному устройству не блокирует обслуживание остальных. Это и есть программная основа параллельного независимого группового управления. **Порядок исполнения:** очередь упорядочивает команды по убыванию приоритета, а при равных приоритетах сохраняет порядок поступления (FIFO) — для этого каждой команде при постановке присваивается метка времени; тем самым обычные команды исполняются в порядке поступления, а высокоприоритетная команда (например, остановка) обгоняет уже ожидающие в очереди.

Прерывание и аварийная остановка. Worker исполняет команду не напрямую, а как отдельную асинхронную подзадачу, ссылку на которую сохраняет; это позволяет прервать выполняемую команду — подзадача отменяется, worker распознаёт отмену, фиксирует её в журнале и немедленно переходит к следующей команде. На этом механизме построена аварийная остановка (E-stop): она очищает очереди всех устройств от ожидающих команд и прерывает исполняемые, после чего драйвер гарантированно переводит устройство в безопасное состояние (см. раздел о потоковой доставке). Важно, что прерывание отделено от отмены: отмена убирает команду из очереди до начала исполнения, тогда как прерывание останавливает уже идущую. **Пауза и возобновление:** с каждым устройством связан признак готовности; когда устройство временно недоступно (например, на время перезапуска его процессов супервизором), worker приостанавливается — уже извлечённая команда удерживается, новые не начинают исполняться, но и не теряются, а после восстановления исполнение продолжается с того же места. **Многоадресные команды:** одна команда может быть адресована сразу

нескольким устройствам — тогда планировщик помещает её в очередь каждого адресата независимо и отслеживает число ещё не завершивших её устройств, сообщая клиенту о завершении лишь после выполнения всеми; это даёт простой и надёжный механизм синхронных групповых действий поверх независимых очередей.

Обобщённый алгоритм работы рабочего процесса устройства приведён на блок-схеме (Рисунок 3.1), а в компактном виде — в Листинге:

```

цикл worker(устройство):
  команда = очередь.извлечь()           # ожидание по приоритету, при равенстве – FIFO
  дождаться готовности устройства       # пауза на время восстановления супервизором
  если команда.отменена:                # отменена, пока ждала в очереди
    пометить_завершённой(команда); продолжить
  подзадача = запустить(драйвер.исполнить(команда)) # отдельная прерываемая задача
  попытка:
    дождаться(подзадача)
  при отмене:                           # прерывание (E-stop) текущей команды
    зафиксировать_прерывание(команда)
  в любом случае:
    пометить_завершённой(команда)     # учёт многоадресных и уведомление ожидающих

```

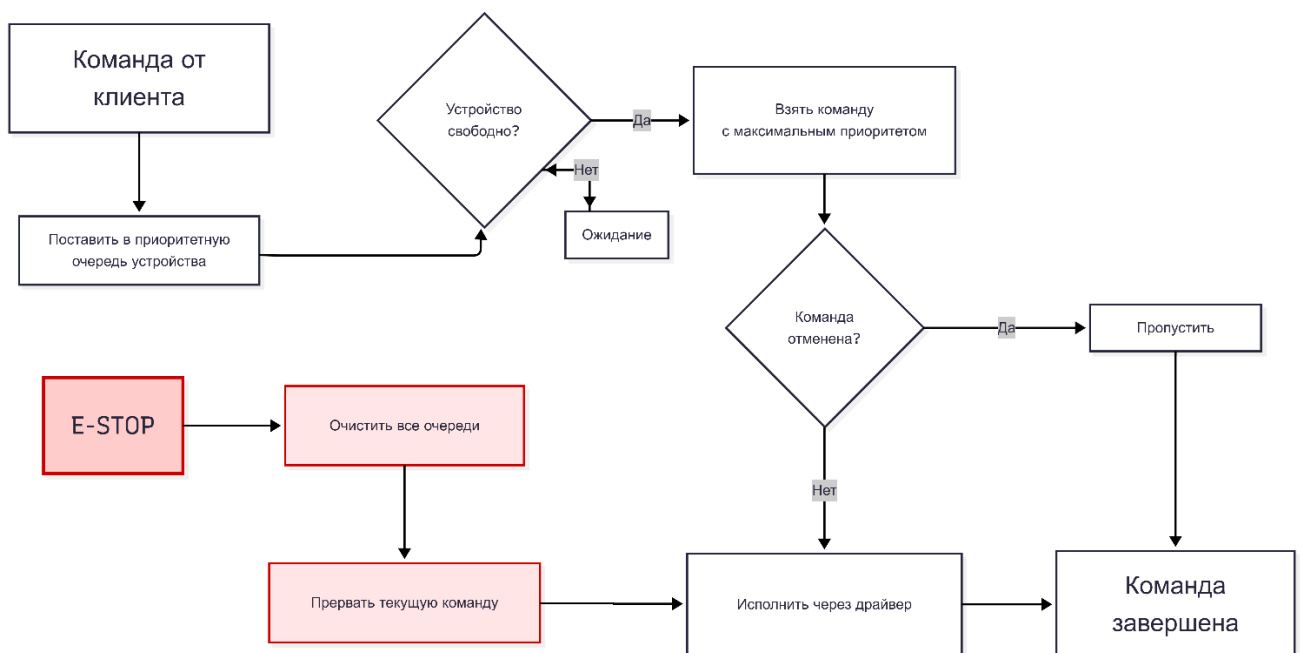


Рисунок 3.1 Блок-схема: очередь команд

Такое разделение «извлечение — ожидание готовности — исполнение как прерываемая подзадача — учёт завершения» позволяет в одном компактном механизме совместить приоритеты, параллелизм по устройствам, аварийное прерывание, паузу на время сбоя и синхронные групповые команды — без явных блокировок и при сохранении строгой последовательности исполнения на каждом устройстве.

3.2. Надёжная потоковая доставка команд и расширение системы

Решаемая задача. Канал связи с устройством не является надёжным: команды и телеметрия проходят через беспроводной участок (WiFi) и протокол rosserial поверх «виртуального» последовательного порта, и при загрузке канала или нестабильном радиосигнале возможны эпизодические потери кадров. Для дискретной команды-события потеря единственного кадра критична: устройство либо не отреагирует на команду движения, либо — что опаснее — не получит команду остановки и продолжит движение. Простой повтор ненадёжен (повтор тоже может потеряться) и не определяет, сколько раз повторять. Поэтому для доставки команд в работе реализован принцип, опирающийся не на передачу события, а на передачу состояния.

Передача состояния вместо события. Вместо однократной отправки команда задаёт целевое состояние исполнительного механизма (например, требуемую скорость), и драйвер повторяет это состояние потоком с фиксированной частотой в течение всего времени действия команды. Потерянный кадр автоматически «исправляется» следующим, приходящим спустя малый интервал: для поддержания состояния не важно, какие именно кадры дошли, важно лишь, что поток не прерывается. Такой подход (повторяющаяся передача целевого значения, setpoint) стандартен для управления мобильными роботами по ненадёжным каналам и обеспечивает устойчивость к потерям без тяжёлых механизмов подтверждения и повторной передачи на уровне каждой команды.

Гарантированная остановка. Особое внимание уделено надёжности именно остановки как наиболее критичной для безопасности операции. По завершении команды движения (либо при её аварийном прерывании) драйвер не просто прекращает поток, а запускает короткий фоновый «стоп-хвост» — поток команд остановки (нулевого целевого состояния), длящийся небольшое время после окончания движения. Стоп-хвост запускается как независимая фоновая задача по принципу «запустил и забыл» (fire-and-forget): благодаря этому он доводится до конца, даже если сама команда была прервана механизмом аварийной остановки. Таким образом, даже в момент E-stop, когда исполнение команды обрывается, остановка устройства всё равно гарантированно передаётся потоком и переживает потери кадров.

Разграничение по актуаторным каналам. Робот несёт несколько независимых исполнительных механизмов (привод колёс, звуковой сигнал, дополнительные реле и сервоприводы), причём команды к части из них передаются через один общий топик. Чтобы поток остановки одного механизма не отменял поток другого, в работе введено

понятие логического актуаторного канала: каждая команда относится к одному каналу, и новая команда сбрасывает фоновый «хвост» только своего канала. За счёт этого, например, продолжающий звучать сигнал не прерывается командой движения, и наоборот. Обобщённая схема доставки показана на Рисунке 3.2, а её ядро — в Листинге.



Рисунок 3.2 Схема потоковой доставки

```

исполнить_длительную(целевое_состояние z, длительность T, канал k):
    отменить_хвост(k)                                # сбросить стоп-хвост этого канала
    старт = now()
    пока now() - старт < T:
        опубликовать(z)                                # повтор целевого состояния
        ждать(1 / частота)
    хвост[k] = запустить_фоном( стоп_хвост(k) ) # fire-and-forget, переживает прерывание

стоп_хвост(k):
    старт = now()
    пока now() - старт < T_стоп:
        опубликовать(нулевое_состояние)                # надёжная остановка потоком
        ждать(1 / частота)
    
```

Описанный механизм применяется единообразно ко всем длительным командам: команда движения повторяет потоком целевую скорость, а по завершении — нулевую; команда звукового сигнала повторяет включение, а затем фоновым хвостом — выключение. Тем самым надёжность доставки обеспечивается на уровне драйвера, прозрачно для остальной системы и для пользователя.

Абстракция драйверов (расширение системы)

Решаемая задача. Одно из ключевых требований к лаборатории — унифицированная работа с разнородными устройствами: пользователь должен управлять любым роботом группы через единый интерфейс, не вникая в то, на каком стеке и по какому протоколу работает конкретное устройство. При этом сама группа гетерогенна: в неё могут входить и роботы на базе ROS, и простые устройства с прямым обменом по последовательному порту, и др. Чтобы совместить единый внешний интерфейс с

разнородностью устройств, в работе разработан драйверный слой — уровень абстракции, переводящий унифицированные команды пользователя в команды, понятные родному контроллеру конкретного устройства. Именно драйверный слой является программным воплощением принципа унификации: разнородность не выносится наружу и не навязывается прошивкам, а скрывается и согласуется на сервере.

Иерархия драйверов. Драйверы организованы в трёхуровневую иерархию. Абстрактный драйвер (`AbstractDriver`) — базовый класс, задающий общий контракт любого драйвера (исполнение команды, запуск необходимых транспортов и адаптеров, инициализация и сворачивание телеметрии, оповещение подписчиков) и не зависящий ни от ROS, ни от конкретного протокола. Интерфейсные драйверы — промежуточные базовые классы, реализующие механику одного класса интерфейсов: `RosBasedDriver` поднимает для устройства транспорт (`socat`) и адаптер (`rosserial`) и даёт доступ к ROS, а `SerialBasedDriver` поднимает только транспорт и работает с байтовым потоком напрямую; на этом уровне фиксируется, «как устройство подключено», но ещё не «что именно оно умеет». Конкретные драйверы — реализуют набор команд конкретной модели устройства, наследуясь от подходящего интерфейсного драйвера; в работе реализованы два таких драйвера: `Yarp13Driver` для робота `yarp-13` (ROS-устройство) и драйвер простого последовательного устройства (обмен ASCII-строками без ROS).

Перевод команды в родной протокол — ядро унификации. Содержательная часть драйвера сосредоточена в методе исполнения команды: он принимает абстрактную команду (имя и аргументы) и формирует из неё конкретное воздействие в терминах родного протокола устройства. Одна и та же по смыслу команда у разных устройств выражается по-разному: для ROS-робота `yarp-13` команда движения превращается в публикацию сообщения скорости в ROS-топик, а специальные команды (сигнал, прямое управление колёсами, сервоприводы) — в публикации в командный топик устройства; для простого последовательного устройства та же команда превращается в текстовую строку фиксированного формата, отправляемую напрямую в порт. Снаружи — для пользователя и для остальных подсистем сервера — оба устройства неотличимы; различие полностью инкапсулировано внутри драйвера. Именно так достигается гетерогенность: поддержка нового класса устройств сводится к написанию драйвера, переводящего унифицированные команды в его родной протокол.

Транспорты и адаптеры как точки расширения. Разнородность проявляется не только в протоколе команд, но и в способе подключения устройства, поэтому драйвер объявляет нужные ему транспорты (доставка байтов) и адаптеры (превращение байтового

потока в высокоуровневый интерфейс). В реализованной системе основной транспорт — socat, основной адаптер — rosserial, но оба звена сменные: для моста на базе ESP-32 принципиально возможны и другие каналы связи (например, Bluetooth), что потребовало бы лишь замены транспорта при неизменных драйвере команд и вышестоящей логике. **Создание драйверов** выполняет фабрика DriverFactory: по указанному в конфигурации типу устройства она создаёт экземпляр соответствующего драйвера, передавая ему общий аппаратный интерфейс нужного рода; добавление нового типа в фабрику — единственная точка, где система «узнаёт» о новом драйвере. Благодаря этому расширение системы локализовано: новое устройство на уже поддержанном интерфейсе требует лишь нового конкретного драйвера, а устройство на новом интерфейсе — дополнительно нового интерфейсного драйвера; во всех случаях изменения вносятся только на сервере и только в драйверном слое, прошивка «тонкого устройства» остаётся неизменной, а планировщик, контроль доступа, супервизор и групповые процедуры продолжают работать с новым устройством единообразно. Это и реализует требование масштабируемости и модульности. Пример прикладного использования драйверов через клиентский интерфейс приведён в скрипте-примере API лаборатории - [Приложение: Демонстрационный код работы с библиотекой RemoteLab].

3.3. Контроль доступа и супервизор устройств

Решаемая задача. Лаборатория рассчитана на одновременную работу нескольких операторов, поэтому необходим механизм разграничения прав на устройства: с одной стороны, дать нескольким пользователям работать параллельно с разными роботами, с другой — исключить конфликт, когда два оператора управляют одним устройством. Для этого реализована подсистема контроля доступа (AccessController).

Модель доступа. Каждое устройство относится к одному из двух режимов, заданному в конфигурации. *Общее (shared)* — устройством может пользоваться любой клиент без предварительного захвата (удобно для неконфликтных сценариев, например наблюдения телеметрии). *Монопольное (exclusive)* — устройство в каждый момент принадлежит не более чем одному клиенту, и чтобы отправлять ему команды, клиент должен сначала его захватить. Подсистема предоставляет операции захвата и освобождения и проверку прав: захват монопольного устройства успешен, только если оно свободно или уже принадлежит данному клиенту, а команда пропускается к исполнению лишь после проверки, что клиент имеет право управлять адресуемым устройством. Тем самым попытка управлять чужим захваченным устройством отклоняется ещё на сервере, до постановки команды в очередь.

Взаимодействие с другими подсистемами. Контроль доступа согласован с групповым характером системы: поскольку очереди и worker'ы устройств независимы, разные операторы, захватившие разные устройства, работают полностью параллельно. Захват ресурсов привязан к клиентской сессии — при отключении клиента все удерживаемые им устройства автоматически освобождаются (см. раздел о сетевом слое), что предотвращает «зависание» ресурсов за отключившимся пользователем. Этот же механизм используется групповыми процедурами, которые захватывают нужные им устройства на время исполнения наравне с обычными клиентами. Контроль доступа реализует требование многопользовательского доступа с разграничением прав и служит одним из элементов модели безопасности системы.

Супервизор устройств

Решаемая задача. Связь с устройствами идёт по нестабильному беспроводному каналу, а представление устройства на сервере обеспечивается набором вспомогательных процессов, которые могут аварийно завершиться (временная потеря связи, перезагрузка робота, сетевой сбой). Чтобы такие отказы не выводили устройство из строя до ручного вмешательства, в работе реализован супервизор устройств (DeviceSupervisor), обеспечивающий автоматическое восстановление. Супервизор ведёт фоновый цикл наблюдения, периодически проверяющий состояние вспомогательных процессов каждого устройства; при обнаружении, что необходимый процесс завершился, запускается процедура восстановления, защищённая от повторного запуска (одновременно для одного устройства выполняется не более одной перезагрузки).

Процедура восстановления согласована с планировщиком и выполняется в строгом порядке: очередь команд устройства приостанавливается (worker переводится в режим ожидания), чтобы на время восстановления никакие команды не уходили в неработающее устройство и при этом не терялись; сворачивается телеметрия и завершаются оставшиеся вспомогательные процессы; заново поднимаются транспорт и адаптер, восстанавливается подписка на телеметрию; очередь возобновляется, и исполнение продолжается с того места, на котором было приостановлено. Связка «пауза очереди на время сбоя — восстановление процессов — возобновление» обеспечивает отказоустойчивость без потери пользовательских команд: команда, пришедшая во время кратковременного сбоя, не отбрасывается, а дожидается восстановления устройства и затем исполняется.

3.4. Групповые процедуры

Серверные групповые процедуры, концептуально описанные в Главе 2, реализованы подсистемой ProcedureManager. Процедура — это исполняемый на сервере асинхронный сценарий, которому доступен весь интерфейс оркестратора: захват и освобождение устройств, постановка команд с ожиданием их физического выполнения, чтение телеметрии, запуск и отмена. Менеджер процедур ведёт реестр запущенных сценариев, изолирует их по доступу (процедура захватывает устройства наравне с обычным клиентом) и обеспечивает их корректное завершение, в том числе принудительную отмену с гарантированной остановкой задействованных роботов. В составе системы реализованы три процедуры: аварийная остановка всей лаборатории (stop_all), групповой тест синхронизации (sync_test — синхронная подача сигнала и разворот всех роботов с проверкой живой телеметрии каждого) и навигационная процедура «По домам» (all_go_home). Ниже подробно рассмотрена последняя как наиболее показательный пример скоординированного группового поведения; её прикладная логика предварительно отработана автором на отдельном симуляторе навигации [7].

Навигационная процедура «По домам»: модель, алгоритмы, реализация

Процедура приводит каждый робот группы из текущей позиции в назначенную ему домашнюю ячейку, объезжая препятствия, при централизованной координации всей группы сервером. Контур управления одного робота образуют четыре связанных компонента — модель движения, оценка состояния (фильтр), планировщик пути и контроллер движения, — объединённые серверным циклом (Рисунок 3.3). Далее каждый из них описан формально.

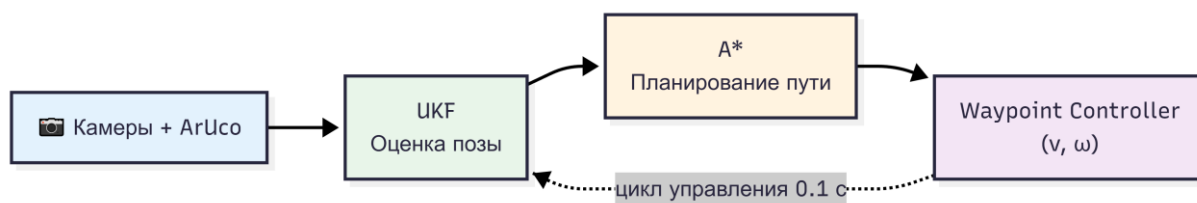


Рисунок 3.3 Концептуальная схема процедуры «По домам»

Модель движения и измерений. Робот описывается кинематической моделью одноколёсника (unicycle) [1] с состоянием $s = [x, y, \theta]$ (координаты и курс) и управлением $u = [v, \omega]$ (линейная и угловая скорости). Дискретная модель движения с шагом Δt :

$$x_{k+1} = x_k + v \cdot \cos(\theta_k) \cdot \Delta t$$

$$y_{k+1} = y_k + v \cdot \sin(\theta_k) \cdot \Delta t$$

$$\theta_{k+1} = \theta_k + \omega \cdot \Delta t$$

Источник измерений — система технического зрения: закреплённые над рабочим полем камеры определяют по ArUco-маркерам глобальную позу каждого робота. Измерение моделируется как зашумлённое наблюдение полного состояния:

$$z = [x + n_x, y + n_y, \theta + n_\theta], \quad n_x, n_y \sim N(0, \sigma_p^2), \quad n_\theta \sim N(0, \sigma_\theta^2)$$

то есть камера наблюдает координаты и курс напрямую, с гауссовым шумом; часть кадров может теряться (перекрытие маркера, пропуск детектирования), и тогда такт коррекции пропускается.

Оценка состояния: сигма-точечный фильтр Калмана (UKF). Измерения зашумлены и эпизодически пропадают, а модель движения нелинейна по курсу, поэтому для каждого робота поток измерений сглаживается сигма-точечным фильтром Калмана (Unscented Kalman Filter, UKF) [6]. UKF аппроксимирует распределение состояния набором $2n+1$ детерминированных сигма-точек (для $n = 3$ — семь точек) и проводит их через нелинейную модель без вычисления якобианов. На каждом такте выполняются шаги предсказания (сигма-точки прогоняются через модель движения по поданному управлению, после чего вычисляются взвешенные среднее и ковариация с добавлением шума процесса Q) и коррекции (по измерению камеры, если кадр получен; ковариация измерения R). Рабочее поле наблюдается несколькими камерами, каждая из которых при удачном кадре даёт собственное измерение позы; эти измерения объединяются в фильтре: на одном такте выполняется один шаг предсказания, а затем — последовательно по одному шагу коррекции на каждое доступное измерение, причём каждое входит со своей ковариацией R (доверием к данной камере). Для независимых шумов последовательные коррекции эквивалентны одной совместной и оптимально взвешивают камеры по их точности; при этом число одновременно работающих камер может меняться (в том числе временно до одной), не нарушая работу контура.

Планирование маршрута: алгоритм A*. Рабочее пространство дискретизируется в сетку ячеек (граф регулярной декомпозиции). Препятствия дополнительно расширяются буферной зоной инфляции (на одну ячейку во все стороны), чтобы маршрут проходил на безопасном удалении от стен. Для каждого робота алгоритм A* [3] строит кратчайший путь от текущей ячейки до домашней по сетке с 4-связностью, используя манхэттенскую эвристику $h(c) = |c_x - goal_x| + |c_y - goal_y|$, допустимую при 4-связности и потому

гарантирующую оптимальность пути. Результат — последовательность ячеек-вейпойнтов от старта до дома; если путь не существует (робот или цель оказались внутри запрещённой зоны), процедура сигнализирует об ошибке для этого робота. В компактном виде алгоритм приведён в Листинге:

```

A*(старт, цель, занятость):
    открытые = { старт };  g[старт] = 0;  f[старт] = h(старт, цель)
    пока открытые не пусты:
        c = узел из открытых с минимальным f
        если c == цель: вернуть восстановить_путь(c)
        открытые.удалить(c)
        для соседа в соседи4(c):
            если занятость[сосед]: продолжить # стена или инфляция
            если g[c] + 1 < g[сосед]:
                пришёл_из[сосед] = c
                g[сосед] = g[c] + 1
                f[сосед] = g[сосед] + h(сосед, цель) # h – манхэттенское расстояние
                открытые.добавить(сосед)
    вернуть «путь не найден»

```

Управление движением: waypoint-контроллер. Построенный маршрут обрабатывается контроллером, ведущим робота последовательно через центры ячеек пути. На каждом такте по текущей оценке позы (x, y, θ) из UKF и координатам целевой ячейки (x_t , y_t) вычисляется ошибка курса:

$$\theta_{target} = \text{atan2}(y_t - y, x_t - x), \quad e_\theta = \text{normalize}(\theta_{target} - \theta),$$

по которой формируется управление по принципу «сначала повернуться на цель, затем ехать»:

$$\omega = \text{clamp}(k \cdot e_\theta, -\omega_{max}, +\omega_{max})$$

$$v = v_{max}, \quad \text{если } |e_\theta| < \varepsilon \quad (\text{робот сориентирован на цель})$$

$$v = v_{max} \cdot \max(0, \cos e_\theta), \quad \text{иначе}$$

$$v = 0, \quad \text{если } |e_\theta| \text{ велико } (> \sim 1 \text{ рад})$$

Угловая скорость пропорциональна ошибке курса (П-регулятор) с ограничением по максимуму; линейная подаётся в полном объёме лишь при достаточно точной ориентации на цель и плавно снижается до нуля при большой угловой ошибке. Целевая ячейка считается пройденной при приближении к её центру ближе порогового радиуса, после чего контроллер переходит к следующей.

Оркестрация на сервере. Перечисленные компоненты объединяет серверный цикл групповой процедуры. Чтобы движущийся робот не становился непредусмотренным (движущимся) препятствием для других, роботы приводятся домой последовательно: пока один едет, остальные неподвижны, и их текущие ячейки добавляются в множество

препятствий при планировании маршрута активного робота. Для каждого робота цикл повторяет: считать свежие измерения камер, выполнить шаг фильтра (предсказание + коррекции), продвинуть планировщик и контроллер и выдать очередную команду движения — до достижения домашней ячейки. Существенно, что управляющие воздействия контроллера выдаются роботу не в обход системы, а через штатный конвейер команд лаборатории: процедура пользуется теми же драйверами и тем же механизмом исполнения, что и одиночные пользовательские команды. Тем самым навигационный контур единообразно встроен в общую архитектуру, а централизованное принятие решений сразу для всей группы (на основе общего поля зрения камер и общей карты препятствий) делает «По домам» примером подлинно коллективного, скоординированного сценария, а не параллельного выполнения независимых команд.

3.5. Телеметрия и сетевой протокол обмена

Помимо отправки команд, система обеспечивает обратный поток — телеметрию состояния устройств. Сбор телеметрии реализован в драйверах: ROS-драйвер подписывается на топики датчиков своего устройства, а драйвер последовательного устройства читает приходящие строки из порта. Полученные «сырые» данные драйвер преобразует в структурированный снимок телеметрии — типизированный набор полей, специфичных для устройства (для робота *uagp-13* это, например, показания одометрии и датчиков расстояния, ориентация, напряжение питания, состояние бамперов и счётчик принятых команд). Приведение разнородных исходных данных к единому структурированному виду — ещё одно проявление унификации: клиент получает телеметрию в одинаковой форме независимо от типа устройства.

Доступ к телеметрии организован в двух режимах. В режиме *push* (*подписка*) клиент регистрирует обработчик, и каждый новый снимок автоматически доставляется ему по мере поступления — этот режим используется для наблюдения в реальном времени. В режиме *pull* (*опрос*) в любой момент доступен последний полученный снимок состояния устройства, без необходимости подписки. Последний снимок поддерживается драйвером всегда, независимо от наличия подписчиков; это позволяет групповым процедурам читать актуальное состояние устройств напрямую, а нескольким клиентам — одновременно наблюдать одно и то же устройство. Доставку телеметрии конкретным клиентам по сети обеспечивает сетевой слой. Реализованный механизм выполняет требование передачи состояния роботов в реальном времени.

Внешний доступ к лаборатории обеспечивает **сетевой слой**, реализованный на базе асинхронного веб-фреймворка (FastAPI/uvicorn). Взаимодействие с клиентом построено поверх одного постоянного соединения WebSocket; формат обмена — текстовые JSON-сообщения с обязательным полем-дискриминатором, по которому сообщение маршрутизируется нужному обработчику. Протокол асинхронный и включает сообщения обоих направлений. Типовой цикл исполнения команды состоит из трёх шагов: клиент отправляет команду → сервер подтверждает её постановку в очередь → по завершении исполнения на устройстве сервер присылает событие завершения (для многоадресной команды — после выполнения всеми адресатами). Телеметрия доставляется в режиме сервер-инициируемой рассылки по подписке. На каждого пользователя поддерживается одна активная сессия. Аутентификация подключения (HTTP Basic Auth с проверкой на этапе WebSocket-рукопожатия и хранением паролей в виде хешей PBKDF2) описана в разделе о безопасности Главы 2.

Клиентская библиотека. Со стороны пользователя работа ведётся через клиентскую библиотеку на Python (RemoteLab), разработанную в настоящей работе и скрывающую детали протокола за высокоуровневым асинхронным интерфейсом. Команда представляется клиенту объектом-дескриптором, который можно либо «выпустить и забыть», либо дождаться его завершения; ожидание многоадресной команды завершается, когда её выполнили все адресаты. Библиотека берёт на себя поддержание соединения: при обрыве связи выполняется автоматическое переподключение, после которого восстанавливаются захваты устройств и подписки на телеметрию, — так что кратковременный сетевой сбой прозрачен для прикладного кода. Корреляция подтверждений с отправленными командами выполняется по порядку их следования в рамках единственного соединения, что гарантирует правильное сопоставление ответов сервера запросам. Выбор формы клиента именно в виде библиотеки, а не готового веб-интерфейса, является сознательным проектным решением: библиотека обеспечивает кроссплатформенность и, что важнее, возможность программной интеграции лаборатории в пользовательские исследовательские и учебные сценарии — от простых скриптов управления до сложных групповых экспериментов. Это напрямую отвечает требованиям доступности, кроссплатформенности и открытости; веб-интерфейс с видеотрансляцией при этом не исключается и может быть построен поверх того же протокола как одно из направлений дальнейшего развития. Типовой сценарий использования библиотеки — от подключения и захвата устройства до отправки команд и чтения телеметрии — приведён в

скрипте-примере API лаборатории [Приложение: Демонстрационный код работы с библиотекой RemoteLab].

Таким образом, в главе описана программная реализация серверного ядра: асинхронная организация ядра, приоритетная диспетчеризация команд с прерыванием и паузой, надёжная потоковая доставка со стоп-хвостом, драйверный слой как воплощение унификации, контроль доступа, супервизор отказоустойчивости, групповые процедуры с навигационным контуром «По домам» (UKF, A*, waypoint-контроллер), телеметрия и сетевой слой с клиентской библиотекой. Совокупно эти механизмы реализуют требования, сформулированные в Главе 1. Далее, в Главе 4, приводятся результаты экспериментальной проверки системы — оценка задержек и устойчивости к потерям, поведение при сбоях и демонстрация групповых сценариев.

Глава 4. ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА СИСТЕМЫ

Предыдущие главы описали архитектуру системы и её программную реализацию. Настоящая глава посвящена экспериментальной проверке работоспособности и характеристик разработанной лаборатории на реальном оборудовании. Сначала описывается собранный экспериментальный стенд и методика испытаний, затем приводятся результаты количественных измерений (задержка управления, устойчивость к потерям пакетов, реакция на разрывы связи) и качественной проверки групповых сценариев — группового теста синхронизации и навигационной процедуры «По домам». Цель главы — показать, что реализованная система действительно обеспечивает удалённое управление реальным роботом в реальном времени и что заложенные в архитектуру механизмы (надёжная доставка, отказоустойчивость, групповая координация на основе технического зрения) работают на практике.

4.1. Экспериментальный стенд

Для испытаний был собран физический полигон (Рисунок 4.1), воспроизводящий рабочие условия виртуальной лаборатории.

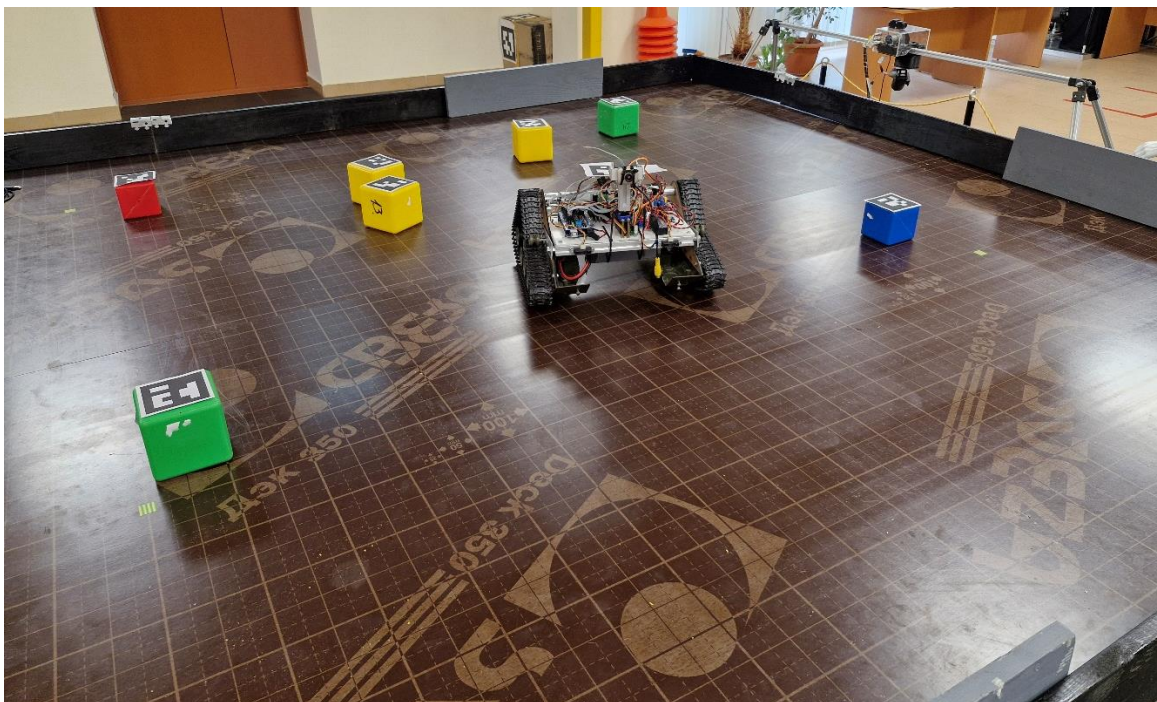


Рисунок 4.1 Экспериментальный стенд

Стенд включает следующие компоненты:

- **Рабочее поле (полигон).** Размеченная горизонтальная площадка с расставленными препятствиями (кубики), ограниченная по периметру; в одной из угловых зон задана

«домашняя» точка робота. Углы рабочего поля обозначены якорными ArUco-маркерами, по которым строится единая система координат полигона.

- **Система технического зрения из двух камер.** Над полигоном закреплены две камеры, каждая из которых видит рабочее поле целиком. Положение и ориентация робота определяются по ArUco-маркеру, размещённому на его корпусе; якорные маркеры в углах задают преобразование «пиксели → метры» (гомография). Использование именно двух камер, наблюдающих весь полигон, позволяет компенсировать шумы, перекрытия маркера и блики и согласуется с подходом, рассмотренным в Главе 1 (потолочные камеры и маркеры в лабораториях Robotarium и Сиены).
- **Робот архитектуры yarp-13.** В качестве управляемого устройства использован мобильный робот дифференциального привода на базе архитектуры yarp-13: низкоуровневый контроллер (Arduino) с клиентом roserial и сетевой мост ESP-32, реализующий прозрачный туннель «UART ↔ TCP». На борту — только эти два компонента, в полном соответствии с принципом «тонкое устройство — умный сервер».
- **Сервер и сеть.** Серверное ядро (RemoteLabManager) развёрнуто на отдельной машине; для каждого устройства поднимаются транспорт (socat) и адаптер (roserial). Связь робота с сервером — по Wi-Fi. Клиент подключается к серверу по WebSocket через клиентскую библиотеку RemoteLab.

Таким образом, собранный стенд содержит все звенья сквозного тракта из Главы 2 — от клиентской библиотеки до исполнительных механизмов робота — и систему позиционирования из двух камер, необходимую для навигационных групповых процедур.



Рисунок 4.1 Экспериментальный стенд в работе

4.2. Количественные и качественные измерения

Количественные измерения характеризуют качество канала и реакцию системы на сбой: средняя задержка «команда → исполнение» и «телеметрия → клиент», доля теряемых кадров на беспроводном участке и время восстановления при разрывах связи

Задержка управления и телеметрии

Задержка — ключевая характеристика для управления в реальном времени. Измерялось время от отправки команды клиентом до её исполнения на роботе и время доставки снимка телеметрии от робота до клиентского callback. В установившемся режиме на лабораторном Wi-Fi средняя сквозная задержка управляющего тракта составила ≈ 20 мс, что для задач телеуправления мобильным роботом субъективно неощутимо и достаточно для замкнутого контура навигационной процедуры, работающей с тактом в десятки миллисекунд. Задержка обратного (телеметрического) тракта оказалась того же порядка. Полученное значение подтверждает пригодность выбранного транспорта (WebSocket поверх постоянного соединения + туннель socat с отключённым алгоритмом Нейгла) для управления в реальном времени.

Устойчивость к потерям пакетов

Беспроводной участок и протокол rosserial поверх виртуального порта подвержены эпизодическим потерям кадров. На стенде доля потерь на беспроводном участке в штатном режиме не превышала $\approx 5\%$. Несмотря на это, управление оставалось

корректным: благодаря принципу потоковой доставки состояния (Глава 3) потерянный кадр автоматически компенсируется следующим, приходящим спустя малый интервал, а критичная команда остановки гарантированно доводится фоновым стоп-хвостом. В отдельной проверке потери искусственно повышались: робот по-прежнему надёжно останавливался по команде E-stop, что подтверждает корректность механизма надёжной остановки на ненадёжном канале. Тем самым показано, что система сохраняет работоспособность и, главное, безопасность при характерном для Wi-Fi уровне потерь.

Отказоустойчивость и реакция на разрывы связи

Отдельно проверялось поведение системы при разрывах — как со стороны устройства, так и со стороны клиента. Разрыв связи с устройством. При обрыве связи с роботом (например, временный уход из зоны уверенного Wi-Fi или перезагрузка) супервизор обнаруживает отказ вспомогательных процессов, приостанавливает очередь команд устройства и выполняет восстановление — заново поднимает транспорт и адаптер и возобновляет исполнение с того же места, не теряя поставленных команд. Полное автоматическое восстановление устройства занимало ≈ 45 с; всё это время команды, пришедшие в адрес устройства, не отбрасывались, а ждали его возвращения. **Разрыв связи с клиентом.** При кратковременном обрыве сетевого соединения клиента библиотека RemoteLab автоматически переподключается и восстанавливает захваты устройств и подписки на телеметрию; восстановление клиентской сессии занимало ≈ 2 с, прозрачно для прикладного кода. При длительном отключении клиента сервер освобождает удерживаемые им устройства, предотвращая «зависание» ресурсов. Эти проверки подтверждают, что заложенные в архитектуру механизмы отказоустойчивости (супервизор устройств и переподключение клиента) работают на реальном стенде.

Групповой тест синхронизации

Для проверки группового тракта реализован и испытан групповой тест синхронизации (процедура `sync_test`): сервер синхронно отдаёт всем участвующим устройствам команды (звуковой сигнал и азворт на месте в обе стороны) и контролирует наличие «живой» телеметрии от каждого. Тест подтверждает, что многоадресные команды доходят до всех адресатов и что система сигнализирует о завершении группового действия только после его выполнения всеми устройствами, а также служит быстрой проверкой исправности всей группы перед запуском более сложных сценариев. На стенде тест проходил корректно: устройства отрабатывали синхронные действия и отдавали телеметрию.

4.3. Навигационная процедура «По домам» для роботов *uarp-13*

Центральный сценарный эксперимент — запуск навигационной процедуры «По домам» (AllGoHome) на собранном полигоне с двумя камерами. В эксперименте участвовал **два робота архитектуры *uarp-13***: на рабочем поле были расставлены препятствия и заданы домашние точки, после чего на сервере была запущена процедура. В ходе исполнения сервер централизованно выполнял весь навигационный контур, описанный в Главе 3: по измерениям **двух камер** (ArUco-маркеры роботов) сигма-точечный фильтр Калмана (UKF) давал устойчивую оценку поз; по дискретной карте полигона с инфляцией препятствий алгоритм A* строил маршруты от текущей ячейки роботов до домашних; *waypoint*-контроллер вёл роботов по маршрутам, а управляющие воздействия выдавались роботу через штатный конвейер команд лаборатории — теми же драйверами, что и обычные пользовательские команды.

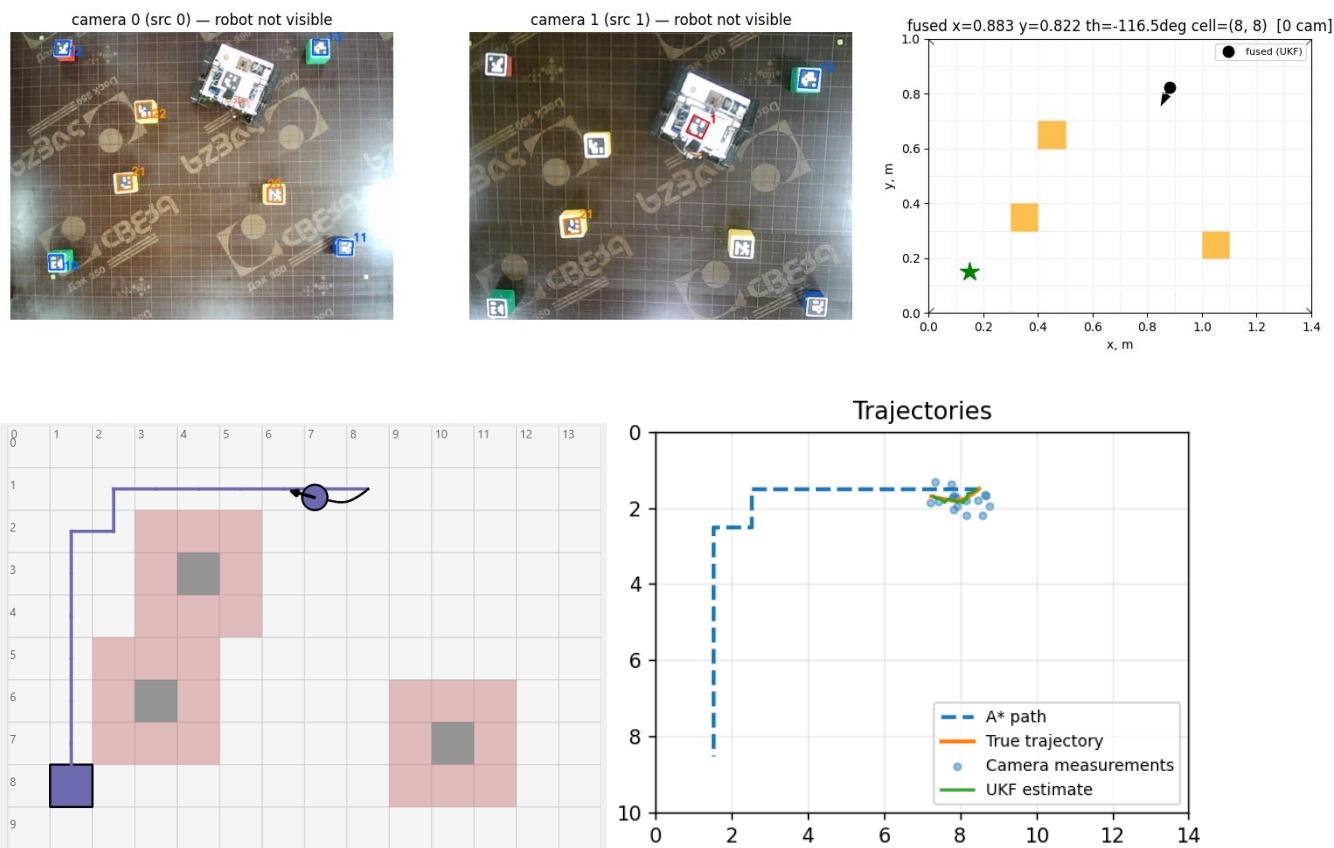


Рисунок 4.2 Тест процедуры «по домам»

Роботы успешно прокладывали маршрут в обход препятствий и доезжали до назначенной домашней точки; на всём протяжении движения оценка позы (по двум камерам) и план маршрута отображались на серверной реконструкции сцены (Рисунок 4.2). Эксперимент подтверждает работоспособность всей вертикали групповой навигации на реальном оборудовании: централизованное восприятие (две камеры + ArUco),

централизованную оценку состояния (UKF-фьюжн) и планирование (A^*), а также исполнение через общий конвейер команд. Тем самым на физическом стенде продемонстрировано, что разрабатываемая система является именно системой управления группой роботов с серверной координацией, а не набором независимо управляемых устройств. Прикладная логика процедуры была предварительно отработана на симуляторе навигации [7], что упростило её перенос на реальный стенд.

4.4. Сводка результатов

Проверяемое свойство	Метод	Результат
Задержка управления (команда → исполнение)	усреднение по серии	≈ 20 мс
Задержка телеметрии (робот → клиент)	усреднение по серии	того же порядка
Потери кадров на Wi-Fi-участке	подсчёт по серии	≤ 5 %
Надёжная остановка при потерях	искусственное повышение потерь + E-stop	остановка гарантирована
Восстановление устройства после разрыва	обрыв связи с роботом	≈ 45 с, без потери команд
Восстановление клиентской сессии	обрыв связи клиента	≈ 2 с, прозрачно
Групповой тест синхронизации	процедура <code>sync_test</code>	пройден
Навигация «По домам» (2 робота uagp-13, 2 камеры)	запуск процедуры AllGoHome на стенде	роботы доходит до дома, обходя препятствия

Таким образом, экспериментальная проверка подтвердила работоспособность системы по всем ключевым направлениям: управление в реальном времени с малой задержкой, устойчивость к характерным для Wi-Fi потерям и гарантированную безопасную остановку, автоматическое восстановление при разрывах связи и, главное, работу групповых серверных сценариев на реальном оборудовании — вплоть до навигационной процедуры «По домам» с позиционированием по двум камерам. Далее, в Заключении, подводятся итоги работы по поставленным задачам, обсуждаются её ограничения и намечаются направления дальнейшего развития.

ЗАКЛЮЧЕНИЕ

В работе разработана система удалённого управления гетерогенной группой роботов в условиях виртуальной лаборатории. Предложена, реализована и экспериментально проверена архитектура «тонкое устройство — умный сервер», в которой вся содержательная логика сосредоточена на центральном сервере, а на борту робота остаётся лишь прозрачный сетевой мост и низкоуровневый контроллер. Поставленная цель — повышение эффективности экспериментальных исследований в области групповой робототехники за счёт системы управления виртуальной лабораторией, учитывающей специфику группового взаимодействия, — достигнута. Поставленные во введении задачи решены в полном объёме.

- **Сравнительный анализ и требования.** Проведён сравнительный анализ существующих решений удалённого управления роботами (rosbridge, Robotarium, Remote Lab Сиены); выявлено, что ни одно из них не сочетает гетерогенность, многопользовательский доступ и открытость интерфейса при работе с группой. На основе анализа сформулированы требования к разрабатываемой системе.
- **Архитектура.** Разработана архитектура виртуальной лаборатории на принципе «тонкое устройство — умный сервер» с поддержкой различных типов устройств и протоколов: драйверный слой переводит унифицированные команды пользователя в команды родного контроллера устройства, а абстракция транспортов и адаптеров снимает привязку к конкретному стеку (ROS) и каналу связи.
- **Алгоритмы и методы управления.** Реализованы ключевые механизмы серверного ядра: приоритетная диспетчеризация команд с аварийным прерыванием и паузой на время восстановления, надёжная потоковая доставка состояния со стоп-хвостом для устойчивости к потерям, контроль доступа для многопользовательской работы, супервизор устройств для отказоустойчивости и менеджер групповых процедур; в составе последних реализована навигационная процедура «По домам» с оценкой состояния (UKF), планированием маршрута (A*) и waypoint-контроллером.
- **Клиентский интерфейс.** Разработана кроссплатформенная клиентская библиотека (RemoteLab) как программный интерфейс доступа к лаборатории: захват устройств, отправка команд с ожиданием выполнения, подписка на телеметрию, запуск групповых процедур, автоматическое переподключение с восстановлением состояния. Поверх того же протокола в дальнейшем может быть построен веб-интерфейс с видеотрансляцией.

- **Испытания.** Собран физический полигон с системой позиционирования из двух камер и проведена серия испытаний на роботах архитектуры uarpr-13: измерены задержка и устойчивость к потерям, проверены отказоустойчивость и групповые сценарии; на стенде запущена навигационная процедура «По домам» для одного робота uarpr-13 с позиционированием по двум камерам. По итогам испытаний система подтвердила работоспособность и может служить основой методического руководства.

Основные результаты. Главный итог работы — действующий программно-аппаратный комплекс, обеспечивающий безопасное удалённое многопользовательское управление гетерогенной группой роботов с серверной групповой координацией. Принципиально новыми по сравнению с рассмотренными аналогами являются сочетание гетерогенности (унификация разнородных устройств драйверным слоем), многопользовательского доступа с разграничением прав и открытого программного интерфейса, а также изначально групповой характер системы, доведённый до работающей навигационной процедуры с централизованным восприятием и координацией на реальном оборудовании.

Ограничения. Качество удалённого управления зависит от характеристик канала связи: на медленном интернет-соединении возрастают задержка управляющего тракта и доля теряемых пакетов, что снижает отзывчивость управления (механизм надёжной потоковой доставки компенсирует потери, но не саму задержку). Кроме того, система не является полностью автономной: для проведения экспериментов ей по-прежнему требуется контроль со стороны персонала лаборатории — в частности, расстановка и обслуживание роботов, подзарядка и надзор за безопасностью рабочей зоны.

Перспективы развития. Естественные направления дальнейшей работы вытекают из заложенной архитектуры. Во-первых, масштабирование группового режима на несколько реальных роботов и расширение парка разнородных устройств. Во-вторых, серверный мониторинг безопасности рабочей зоны, используя уже имеющуюся централизованную картину поз группы (камеры + ArUco). В-третьих, построение веб-интерфейса с видеотрансляцией поверх существующего протокола для снижения порога входа. В-четвёртых, разработка автоматических станций зарядки на базе уже реализованной процедуры «По домам», что напрямую уменьшает участие персонала в обслуживании парка. В совокупности эти направления развивают систему в сторону полнофункциональной групповой удалённой лаборатории, сохраняя её ключевые принципы — централизацию логики, унификацию разнородных устройств и безопасность удалённого доступа.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Adzkiya D. [и др.]. Control design of discrete-time unicycle model using satisfiability modulo theory // Systems Science and Control Engineering. 2024. № 1 (12).
2. AlQahtani A. A. S. Key Derivation: A Dynamic PBKDF2 Model for Modern Cryptographic Systems // Cryptography. 2025. № 2 (9).
3. Bundy A., Wallen L. A* Algorithm 1984.
4. Casini M. [и др.]. A remote lab for experiments with a team of mobile robots // Sensors (Switzerland). 2014. № 9 (14).
5. Lee J. Web Applications for Robots using rosbridge [Электронный ресурс]. URL: https://cs.brown.edu/media/filer_public/b0/03/b0037d43-76b6-4729-8c1e-635f120e5192/lee.pdf (дата обращения: 18.06.2026).
6. Luo X., Moroz I. M. Ensemble Kalman filter with the unscented transform // Physica D: Nonlinear Phenomena. 2009. № 5 (238).
7. Nepike ALLGOHOME — симулятор навигации мобильных роботов [Электронный ресурс]. URL: <https://github.com/Nepike/allgohome> (дата обращения: 18.06.2026).
8. Pickem D. [и др.]. The Robotarium: A remotely accessible swarm robotics research testbed 2017.
9. Quigley M. [и др.]. ROS: an open-source Robot Operating System 2009.

ПРИЛОЖЕНИЯ

Низкоуровневый протокол связи

Создание архитектуры удаленного управления роботами началось с попытки создать собственный протокол взаимодействия между ESP-8266 и Arduino. Плата ESP выбрана из тех соображений, что для того, чтобы робот мог подключаться к сети и получать команды через интернет ему нужен какой-то модуль WIFI. ESP же как раз содержит такой модуль и кроме того способна производить несложные вычисления, которые будут основой протокола.

Таким образом, наша цель – разработать систему управления периферийными устройствами (например машинкой или манипулятором) через микроконтроллер ESP8266, получая команды удалённо (через интернет). Ключевые требования:

- **Гибкость:** Возможность отправлять команды с произвольным количеством аргументов различных типов (bool, int, float, string).
- **Единообразие:** Одинаковый формат обмена для всех устройств, независимо от функционала.

ESP8266 выступает в роли “мозга” системы, преобразуя команды (например, в формате json) в низкоуровневые инструкции Arduino. Последние отвечают за выполнение действий и отправку данных обратно в интернет через ESP.

Структура пакета

Предлагается использовать такую структуру пакета:

[Start Marker][Command][Data Type Flags][Payload Length][Payload][CRC]

- **Start Marker (2 байта):** Уникальная последовательность (например, 0xAA55), необходимая для синхронизации приемника и защиты от мусора в UART.
- **Command (1 байт):** Идентификатор команды (например, 0x01 – движение манипулятора)
- **Data Type Flags (N байт):** Битовая маска типов данных аргументов в payload. Формат(байты): [кол-во аргументов][тип 1][тип 2]...[тип N].

Типы предлагается закодировать подобным образом:

Тип данных	Код	Размер (байт)	Пример значения
bool	0x01	1	0x01 (true)
uint8_t	0x02	1	0x64 (100)
int16_t	0x03	2	0x00 0x96 (150) (<i>big-endian</i>)
uint16_t	0x04	2	0x003 0xE8 (1000)
float	0x05	4	0x42 0x36 0x00 0x00 (45.5) (<i>IEEE754 float</i>)

char[]	0x06	N	0x05 0x48 0x65 0x6C 0x6C 0x6F (“Hello”)
--------	------	---	---

Пример для [bool, int16, float]: [0x03 0x01 0x03 0x05]

- **Payload Length (1 байт):** Общий размер аргументов в байтах.
- **Payload (N байт):** Непосредственные значения аргументов, упакованные последовательно.
- **CRC (2 байта):** Контрольная сумма для проверки целостности данных (алгоритм CRC-16).

Алгоритм кодирования (на стороне ESP)

Разберем алгоритм на примере абстрактной команды set_position(true, 150, 45.5)

1. Прием команды из интернета (например, JSON):

```
{“cmd”: “SET_POSITION”,
“params”: [true, 150, 45.5]}
```

2. Определение идентификатора команды:

“SET_POSITION” → 0x02 (например)

3. Определение типа данных:

true → bool (0x01)

150 → int16 (0x03)

45.5 → float (0x05)

Data Type Flags: 0x03 0x01 0x03 0x05

4. Упаковка Payload:

true → 0x01

150 → 0x00 0x96

45.5 → 0x42 0x36 0x00 0x00

5. Расчет CRC для всего пакета.

Алгоритм декодирования (на стороне Arduino) – очевиден (последовательное чтение данных согласно правилам записи).

Сравнение с Modbus

Для сравнения рассмотрим стандартизированный промышленный протокол связи - Modbus. Рассмотрим важные особенности этого протокола:

- **Фиксированные типы данных:** Только 16-битные регистры и одноканальные coils.

- **Фиксированная структура пакета:** Функции представляют из себя только чтение/запись регистров и битовые операции.
- **Slave-устройство должно знать структуру регистров:** Добавление нового параметра требует изменения прошивки Arduino.

Эти особенности создают проблемы для данного проекта: из-за жесткой структуры невозможно передать переменное число аргументов в одной команде, к тому же, совсем непонятно как работать с типами данных, занимающими больше 16 бит (например float или строки).

Наш протокол решает эти фундаментальные проблемы, однако содержит другие слабые стороны, которые в конечном итоге заставят искать другое решение...

Слабые стороны протокола

Предложенный низкоуровневый протокол, несмотря на достижение целей гибкости и единообразия, обладает рядом существенных недостатков, ограничивающих его применение в масштабируемых и долгосрочных проектах. Критический анализ выявляет следующие слабые стороны:

1. **Отсутствие механизма расширяемости и динамического согласования.** Архитектура протокола является статической. Для введения новой команды или модификации необходимо вносить изменения в программное обеспечение как на стороне передатчика (ESP8266), так и на стороне приемника (Arduino). Это требует одновременного обновления прошивок всего парка устройств, что делает процесс эволюции системы трудоемким и подверженным ошибкам. В отличие от стандартизированных протоколов, где функции могут быть запрошены динамически, данный протокол требует жесткой, заранее определенной таблицы соответствия между идентификаторами команд и их семантикой.
2. **Отсутствие стандартизации и совместимости.** Будучи проприетарным решением, протокол несовместим с существующими промышленными стандартами (такими как Modbus RTU, хотя он и был отвергнут по причине ограниченности типов данных). Это создает технологический вакуум вокруг системы, усложняя ее интеграцию в более крупные экосистемы.
3. **Потенциальная неэффективность использования пропускной способности.** Использование многобайтовых преамбул (таких как маркер начала и флаги типов) для каждого короткого пакета может привести к значительным накладным расходам, особенно при частой передаче команд с малым объемом полезной нагрузки.

4. **Ограниченные возможности для отладки и диагностики.** Протокол не предусматривает обязательных полей для идентификации устройства-отправителя, последовательного номера пакета или статуса выполнения команды. Это затрудняет диагностику проблем в сети, состоящей из нескольких устройств, и не позволяет реализовать надежные механизмы подтверждения доставки и повторной передачи.

Таким образом, хотя предложенный протокол успешно решает задачу гибкой передачи типизированных данных, его фундаментальные ограничения, связанные со статичностью, высокими затратами на поддержку и отсутствием стандартизации, делают его непригодным для проектов, требующих долгосрочного развития и масштабирования. Эти недостатки являются веским основанием для рассмотрения альтернативных, более зрелых решений.

Несмотря на это «сырая» версия протокола была реализована и протестирована. Полный код алгоритма, а также примеров кода для master и slave можно найти в директории `progs/custom_protocol`.

Полный код ESP-32 (мост TCP ↔ UART)

```
/*
 * esp32gateway – прозрачный сетевой мост «TCP ↔ UART» для робота.
 *
 * Роль в системе.
 * Это прошивка «тонкого устройства»: ESP32 не интерпретирует данные и не
 * содержит никакой логики управления роботом. Его единственная задача –
 * двусторонне перекладывать поток байтов между TCP-соединением (по WiFi
 * с центральным сервером лаборатории) и аппаратным UART, к которому
 * подключён
 * низкоуровневый контроллер робота (Arduino с клиентом rosserial).
 *
 * С точки зрения сервера всё выглядит так, будто Arduino подключён к нему
 * напрямую по последовательному кабелю.
 *
 * Прошивка намеренно тривиальна и НЕ меняется при добавлении новых команд
 * или
 * типов устройств – она лишь передаёт байты.
 */

#include <WiFi.h>    // Стек WiFi для ESP32 (TCP/IP, режим станции)

// --- Параметры подключения к точке доступа WiFi ---
const char* ssid     = "WIFI-NAME";
const char* password = "WIFI-PASSWORD";

// TCP-сервер слушает порт 2000 – к нему подключается socat со стороны
сервера
// лаборатории (порт должен совпадать с полем "port" устройства в
devices.json).
WiFiServer server(2000);
```



```

// Текущее клиентское соединение. Мост рассчитан на одного клиента
одновременно:
// единственный клиент — это сервер лаборатории (socat).
WiFiClient client;

// --- Распиновка ---
#define RXD2 16          // UART2 RX (приём от Arduino)
#define TXD2 17          // UART2 TX (передача в Arduino)
#define LED_PIN 2        // Встроенный светодиод — индикация состояния

// --- Буфер для пакетной отправки в TCP ---
// Байты, пришедшие из UART, копятся здесь и отправляются в TCP пачками, а не
// по одному. Это снижает накладные расходы: вместо множества крошечных
// TCP-сегментов (заголовок ~40 байт на каждый) отправляется один пакет.
#define TCP_BUF_SIZE 64
uint8_t tcpBuffer[TCP_BUF_SIZE]; // сам буфер накопления
uint8_t tcpBufPos = 0;           // сколько байт уже накоплено (позиция
записи)

void setup() {
    // Отладочная консоль по USB (логи подключения, IP-адрес). На обмен с
    роботом
    // не влияет — это отдельный последовательный порт (UART0).
    Serial.begin(115200);

    // Светодиод как индикатор: горит — WiFi подключён, мигает — идёт
    подключение.
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    // Отключаем энергосбережение WiFi. По умолчанию модем периодически
    «засыпает»,
    // что добавляет задержки и провоцирует обрывы — для прозрачного туннеля в
    // реальном времени это критично, поэтому держим радио всегда активным.
    WiFi.setSleep(false);
    WiFi.begin(ssid, password);

    // Ждём установления соединения с точкой доступа, мигая светодиодом.
    Serial.print("Connecting to WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));
    }

    // Подключились — выводим полученный IP (его указывают в devices.json как
    "ip").
    Serial.println("\nConnected!");
    Serial.print("ESP32 IP: ");
    Serial.println(WiFi.localIP());
    digitalWrite(LED_PIN, HIGH);

    server.begin();

    // Инициализируем UART2 для связи с Arduino.
    // 57600 бод, 8 бит данных, без чётности, 1 стоп-бит (8N1).
    // ВАЖНО: скорость должна совпадать со скоростью низкоуровневого
    контроллера
    // (и параметром baud у rosserial на сервере), иначе обмен
    рассинхронизируется.
    Serial2.begin(57600, SERIAL_8N1, RXD2, TXD2);
}

```

```

void loop() {
    // Если активного клиента нет (ещё не подключился или соединение разорвано)
    -
    // пытаемся принять новое входящее соединение.
    if (!client || !client.connected()) {
        WiFiClient newClient = server.available();
        if (newClient) {
            client = newClient;
            // Отключаем алгоритм Нейгла (TCP_NODELAY): мелкие пакеты отправляются
            // немедленно, без ожидания накопления. Для управления в реальном
времени
            // важнее низкая задержка, чем эффективность канала.
            client.setNoDelay(true);
            Serial.println("Client connected");

            // Сбрасываем «хвосты» от предыдущей сессии, чтобы новая началась с
чистого
            // листа: опустошаем приёмный буфер UART, приёмный буфер TCP и обнуляем
            // буфер накопления. Иначе устаревшие байты могли бы сбить кадрирование
            // протокола roserial.
            while (Serial2.available()) Serial2.read();
            while (client.available()) client.read();
            tcpBufPos = 0;
        }
        yield(); // отдаём управление планировщику/служебным задачам ESP32
        return; // пока клиента нет — пересылать нечего, выходим из итерации
    }

    // Пока в TCP есть данные И в передающем буфере UART есть место —
пересылаем
    // байт за байтом. Проверка availableForWrite() не даёт заблокироваться,
если
    // UART не успевает отправлять (Arduino медленнее, чем сеть).
    while (client.available() && Serial2.availableForWrite() > 0) {
        Serial2.write(client.read());
    }

    // Считываем доступные байты из UART в буфер накопления.
    while (Serial2.available()) {
        if (!client.connected()) break; // клиент отвалился — прекращаем чтение

        uint8_t c = Serial2.read();
        tcpBuffer[tcpBufPos++] = c;

        // Буфер заполнился — отправляем пачку в TCP и освобождаем его.
        if (tcpBufPos >= TCP_BUF_SIZE) {
            flushTcpBuffer();
        }
    }

    // Если в буфере остались байты, но больше ничего не приходит ни из TCP, ни
из
    // UART, — отправляем их немедленно, не дожидаясь заполнения буфера. Это
    // ограничивает задержку: короткие сообщения не «зависают» в ожидании ещё
    // данных.
    if (tcpBufPos > 0 && !client.available() && !Serial2.available()) {
        flushTcpBuffer();
    }

    yield(); // обязательная уступка: даём ESP32 обслужить WiFi-стек и
watchdog
}

```

```
// Отправка накопленного буфера в TCP-соединение.
void flushTcpBuffer() {
    if (tcpBufPos > 0 && client.connected()) {
        client.write(tcpBuffer, tcpBufPos);    // отправляем накопленные байты
        пачкой
        client.flush();                        // гарантируем отправку
        (проталкиваем из буфера сокета)
        tcpBufPos = 0;                        // буфер опустошён — начинаем
        накопление заново
    }
}
```

Демонстрационный код работы с библиотекой RemoteLab

```
"""
example.py — путеводитель по возможностям виртуальной лаборатории RemoteLab.
"""

import asyncio
import os
import sys

from client import (
    RemoteLab,
    AcquireError,
    SubmitError,
    ProcedureError,
    ConnectionLostError,
)

# ===== Настройки =====
SERVER_URL = "localhost:8000"    # можно "ws://host:port" или просто
                                "host:port"
USERNAME   = "test"
PASSWORD   = "1234"

# ===== ДЕМО-СЦЕНАРИИ =====
# Каждый сценарий самодостаточен и снабжён пояснением. Можно закомментировать
# ненужные вызовы в main().

async def demo_telemetry(lab: RemoteLab, device_names: list):
    """
    Телеметрия двумя способами:
    push — подписка с callback (вызывается на каждое новое сообщение);
    pull — разовое чтение последнего снимка get_latest_telemetry().
    """
    print("[Телеметрия] подписываемся на push-поток первого устройства...")
    name = device_names[0]
    robot = lab.device(name)

    # callback получает обычный dict с полями телеметрии (см.
    DRIVER_TELEMETRY)
```

```

await robot.subscribe_telemetry(lambda data: None) # тихо копируем снимки

await asyncio.sleep(1.0) # дать потоку прийти

snap = robot.get_latest_telemetry() # pull: последний снимок без
callback
if snap is None:
    print(f" ⚠ от '{name}' пока нет телеметрии")
else:
    # показываем пару характерных полей (если есть)
    keys = [k for k in ("compass", "acc_voltage", "cmd_count") if k in
snap]
    shown = ", ".join(f"{k}={snap[k]}" for k in keys) or "снимок получен"
    print(f" '{name}': {shown}")
print()

async def demo_single_device(lab: RemoteLab, devices: list):
    """
    Одиночные команды одному роботу + правильная работа с эксклюзивным
    доступом.

    lock() безопасен и для shared-устройств (там это по-оп), поэтому шаблон
    `async with robot.lock()` универсален.
    """
    d = devices[0]
    name = d["name"]
    robot = lab.device(name)
    print(f"[Одиночное устройство] работаем с '{name}'...")

    # lock() гарантирует release даже при исключении
    async with robot.lock():
        # CommandHandle можно дожидаться (await) либо «выстрелить и забыть»
        print("  beep 0.5с (ждём завершения)...")
        cmd = await robot.submit("beep", duration=0.5)
        await cmd # блокируемся до
        # выполнения

        print("  короткий поворот на месте (fire-and-forget + await)...")
        cmd = await robot.submit("move", speed_lin=0.0, speed_ang=1.0,
duration=1.0)
        await cmd
        # на всякий случай явный стоп (потокостановка на сервере и так
        # есть)
        stop = await robot.submit("stop", priority=9)
        await stop
        print()

async def demo_group_direct(lab: RemoteLab, devices: list):
    """
    Групповая команда НАПРЯМУЮ с клиента: одна submit на список устройств.
    Хэндл завершится, когда КАЖДОЕ устройство отчитается 'done'.

    Берём только shared-устройства: для exclusive нужен предварительный lock
    каждого (см. demo_single_device).
    """
    shared = [d["name"] for d in devices if d["shared"]]
    if not shared:
        print("[Группа] нет shared-устройств для прямой групповой
команды.\n")
        return

    print(f"[Группа] синхронный beep на {shared}...")

```

```

cmd = await lab.submit(shared, "beep", priority=5, duration=0.7)
await cmd
print(" готово.\n")

async def demo_procedures(lab: RemoteLab):
    """
    Серверные групповые ПРОЦЕДУРЫ – сценарии, исполняемые на сервере целиком.
    Клиент лишь запускает их по имени и ждёт результат.

    Зарегистрированные процедуры:
    sync_test    – синхронный тест-парад всей группы (пищат, крутятся влево/
                  вправо) + проверка живой телеметрии каждого робота;
    stop_all     – аварийная остановка всей лаборатории;
    all_go_home  – возврат роботов на базу (камера + ArUco + A*;) .

    Важно: процедура сама захватывает нужные устройства, поэтому НЕ держите
на
клиенте lock на те же устройства во время её запуска.
    """
    print("[Процедура] запускаем sync_test (beep → влево 3с → вправо 3с →
    проверка телеметрии)...")
    try:
        proc = await lab.run_procedure("sync_test")
        await proc # ждём завершения; при провале телеметрии бросит
        ProcedureError
        print(" sync_test: УСПЕХ – все роботы живы и синхронны.\n")
    except ProcedureError as e:
        print(f" sync_test: ПРОВАЛ – {e}\n")

async def demo_emergency_stop(lab: RemoteLab, device_names: list):
    """
    Шаблон аварийной остановки одного устройства:
    1. interrupt() – прерывает команду, выполняемую ПРЯМО СЕЙЧАС;
    2. submit("stop", priority=...) – высокоприоритетный стоп следующим.
    Для остановки ВСЕЙ лаборатории используйте процедуру stop_all.
    """
    name = device_names[0]
    robot = lab.device(name)
    print(f"[Авария] демонстрация interrupt+stop на '{name}'...")

    # запускаем длинное движение и тут же прерываем его
    await robot.submit("move", speed_lin=0.0, speed_ang=1.0, duration=10.0)
    await asyncio.sleep(0.5)
    await robot.interrupt() # прервать текущее
    await robot.submit("stop", priority=999) # и сразу стоп
    print(" движение прервано и остановлено.\n")

# ===== MAIN
=====

async def main():
    print(f"Подключаемся к {SERVER_URL} под пользователем '{USERNAME}'...\n")
    try:
        async with RemoteLab(SERVER_URL, USERNAME, PASSWORD) as lab:
            print("Подключено.\n")

            # 1. Какие устройства есть и что им можно слать
            devices = await lab.get_devices()
            if not devices:
                print("Активных устройств нет – нечего демонстрировать.")
                return
            print_devices(devices)
            print_command_catalog(devices)

```

```

device_names = [d["name"] for d in devices]

# 2. Телеметрия (push + pull)
await demo_telemetry(lab, device_names)

# 3. Одиночные команды + эксклюзивный доступ
await demo_single_device(lab, devices)

# 4. Прямая групповая команда с клиента
await demo_group_direct(lab, devices)

# 5. Серверная групповая процедура (синхронный тест группы)
await demo_procedures(lab)

# 6. Аварийная остановка (interrupt + stop)
await demo_emergency_stop(lab, device_names)

print("Все сценарии отработаны.")

except AcquireError as e:
    print(f"Не удалось захватить устройство: {e}")
    sys.exit(1)
except SubmitError as e:
    print(f"Сервер отклонил команду: {e}")
    sys.exit(1)
except ConnectionLostError as e:
    print(f"\nCоединение потеряно: {e}")
    sys.exit(1)
except OSError as e:
    print(f"\nНе удалось подключиться к серверу ({SERVER_URL}): {e}")
    sys.exit(1)

if __name__ == "__main__":
    asyncio.run(main())

```